

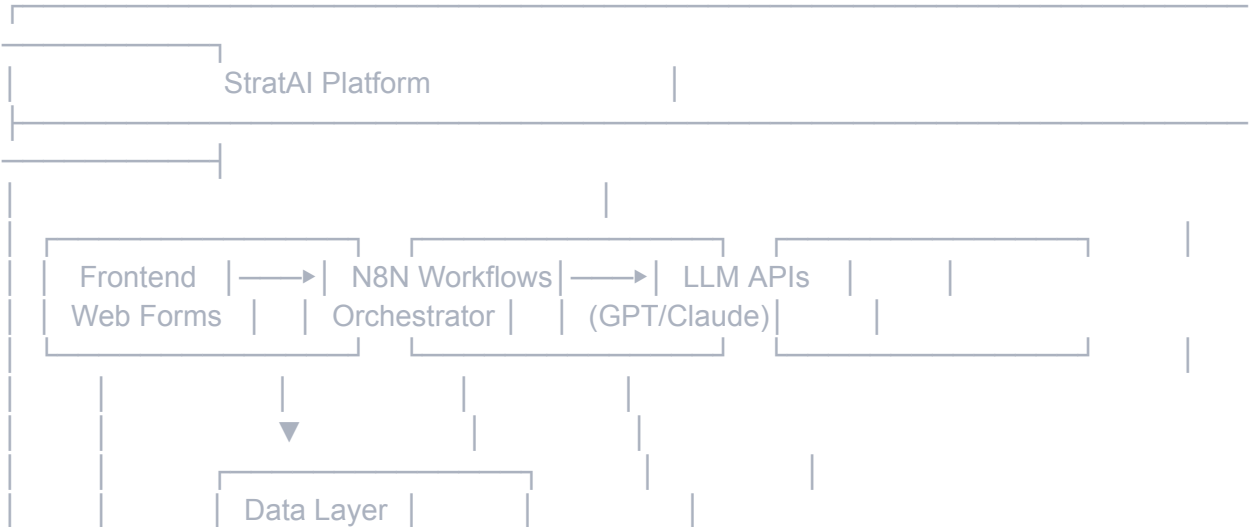
StratAI - Technical Architecture & Documentation

Table of Contents

- 1. System Overview
- 2. Architecture Diagrams
- 3. Component Specifications
- 4. Data Flow
- 5. API Documentation
- 6. Security & Compliance
- 7. Deployment Guide
- 8. Monitoring & Observability
- 9. Disaster Recovery
- 10. Performance Optimization

1. System Overview

1.1 High-Level Architecture





1.2 Technology Stack

Layer	Technology	Purpose	Version
Orchestration	N8N	Workflow automation & agent coordination	Latest
LLM Processing	OpenAI GPT-4o	Multi-agent analysis (5 agents)	gpt-4o
LLM Chat	Anthropic Claude 3.5 Sonnet	Interactive Q&A	claude-3-5-sonnet
Database	Airtable	Structured data storage	API v0.3
Document Gen	Google Docs API	Report generation	v1
File Storage	Google Drive API	PDF storage & retrieval	v3
Email	Gmail API	Notifications & digests	v1
Communication	Slack API / MS Teams Webhooks	Team notifications	Latest
Monitoring	N8N Logs + Airtable Analytics	System observability	-

1.3 System Components

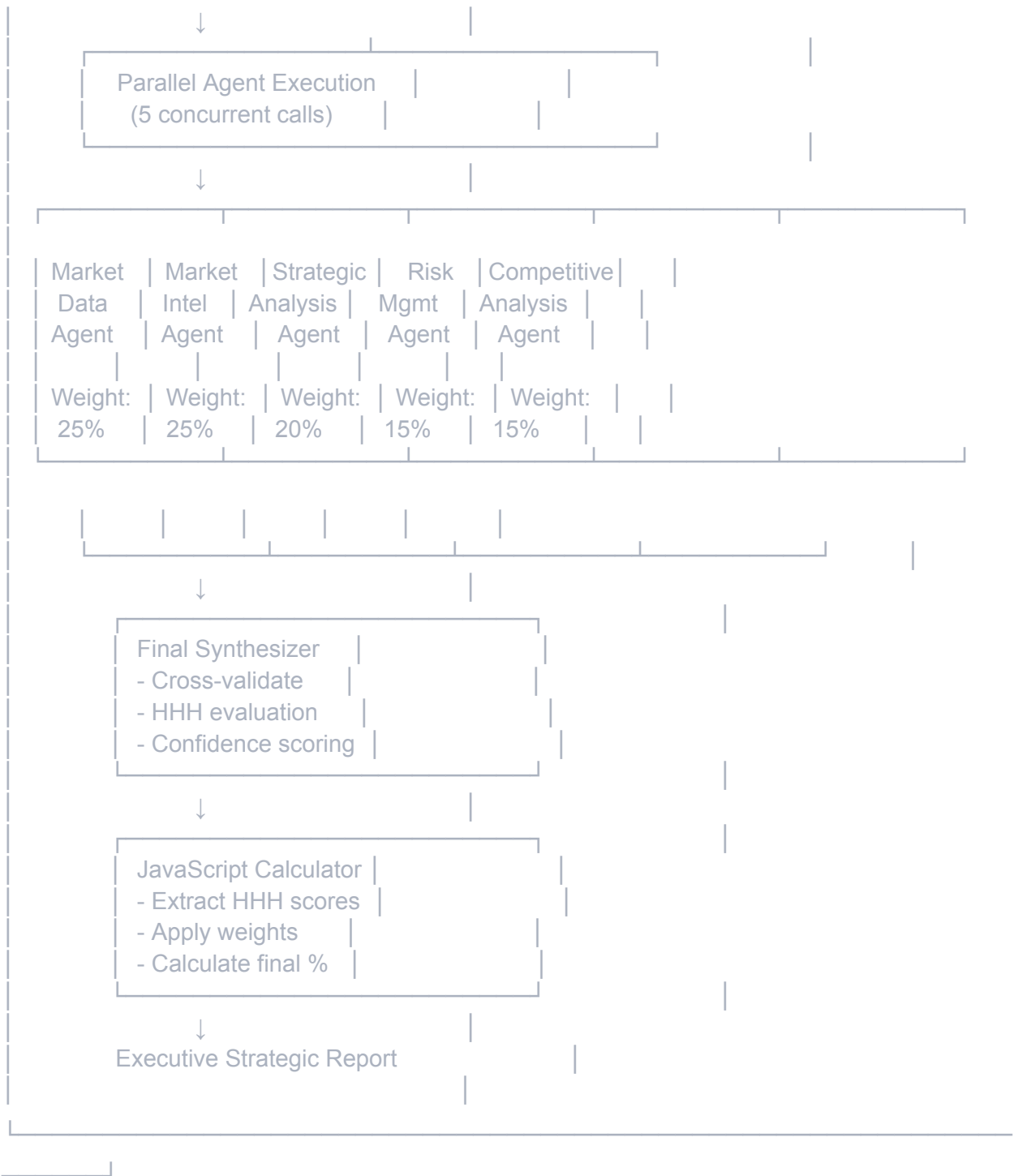
mermaid
graph TB

A[User Interface] --> B[Webhook Endpoints]
B --> C[Data Extraction Layer]
C --> D[PDF Parser]
C --> E[Form Validator]
E --> F[Multi-Agent Ensemble]
F --> G1[Market Data Agent]
F --> G2[Market Intelligence Agent]
F --> G3[Strategic Analysis Agent]
F --> G4[Risk Management Agent]
F --> G5[Competitive Analysis Agent]
G1 --> H[Final Synthesizer]
G2 --> H
G3 --> H
G4 --> H
G5 --> H
H --> I[Confidence Calculator]
I --> J[Report Generator]
J --> K[Google Docs]
J --> L[Airtable Storage]
J --> M[Notification Router]
M --> N1[Email]
M --> N2[Slack]
M --> N3[Teams]

2. Architecture Diagrams

2.1 Multi-Agent Architecture





2.2 Data Flow Architecture



User Submission



Webhook	
/stratai-	
submit	



Extract Form Data (Code Node)	
- user_email	
- company_name	
- strategic_question	
- market_intelligence	
- strategic_context	
- risk_factors	
- competitive_landscape	
- organizational_insights	
- user_ai_chip_rate (1-100)	
- user_crypto_rate (1-100)	
- user_ev_rate (1-100)	
- user_supply_risk (1-100)	
- user_regulatory (1-100)	
- user_consumer_rate (1-100)	
- pdf_file (optional)	
- session_id (generated)	



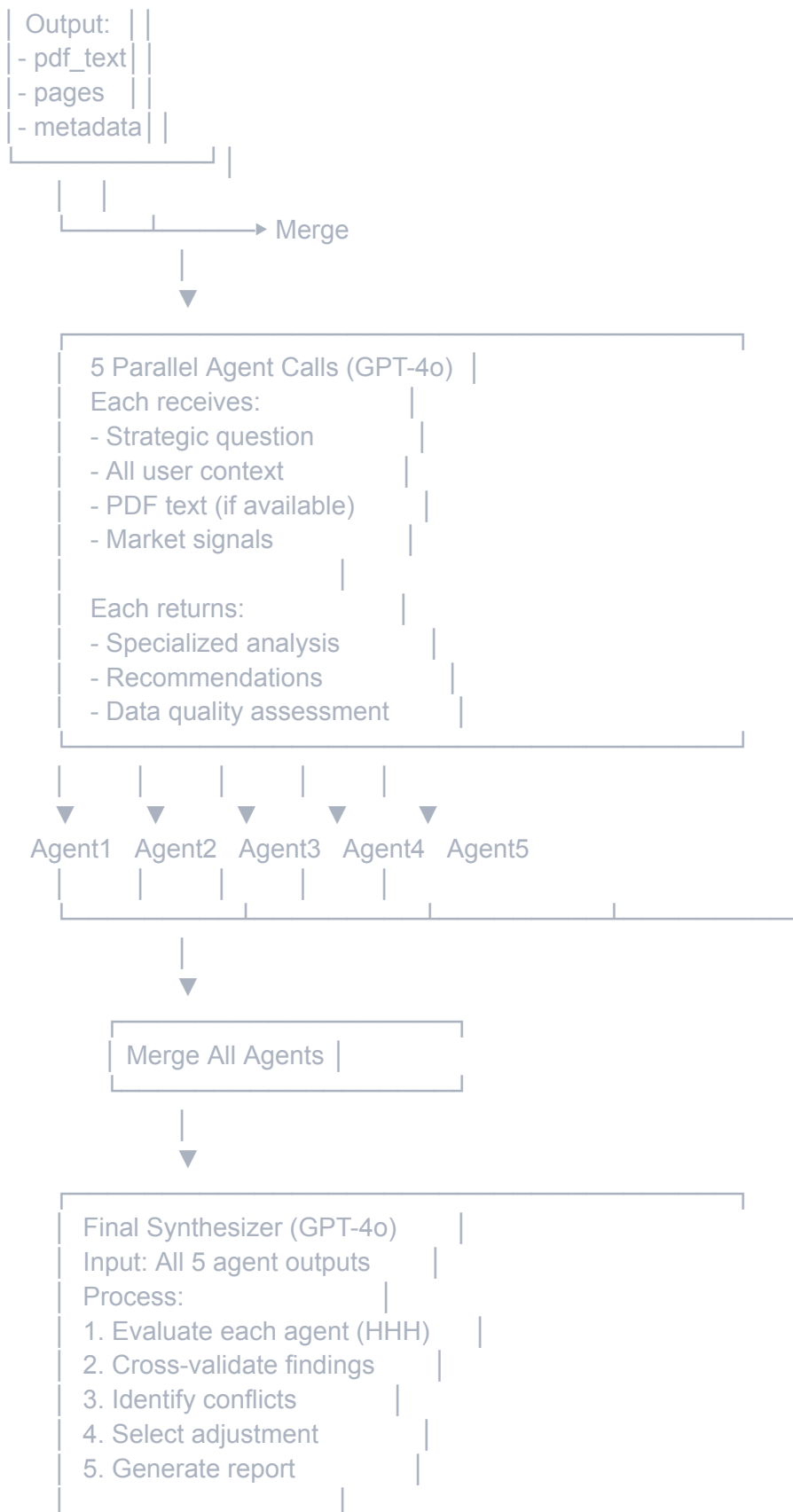
IF: PDF?	
----------	--



YES	NO	
-----	----	--



PDF Parser		Skip	



Output: Full report with:

- HHH scores for each agent
- Base confidence calculation
- Selected adjustment (+8/0/-7)



Extract & Validate (JavaScript)

Process:

1. Parse synthesizer output
2. Extract HHH scores via regex
3. Calculate base score:
 $(MD \times 0.25) + (MI \times 0.25) + (SA \times 0.20) + (RM \times 0.15) + (CA \times 0.15)$
4. Detect adjustment from text
5. Compute: base + adjustment
6. Extract report sections

Output: Structured JSON with

- confidence_score (calculated)
- executive_summary
- recommendations
- all report sections

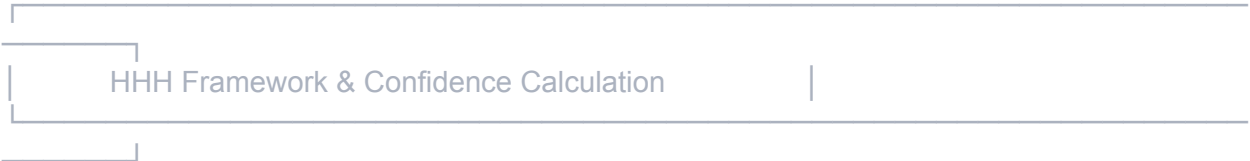


Create Google Doc	Save to Airtable
- Full report	- report_id
- Formatted MD	- confidence
	- all metadata
Returns:	
- documentUrl	

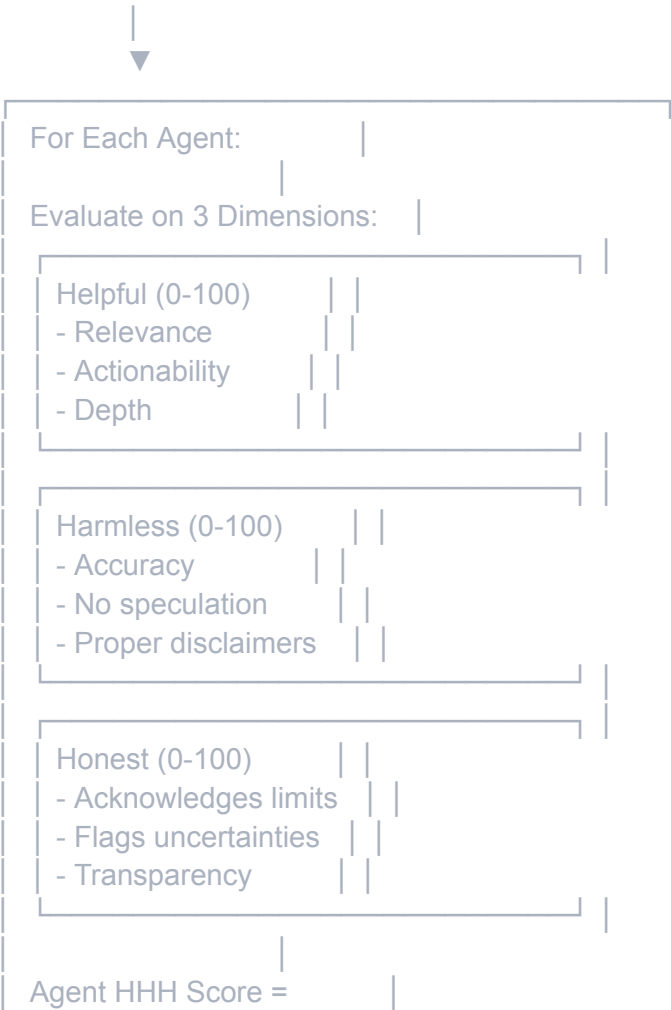


```
Respond to Webhook
{
  success: true,
  report_id,
  confidence_score,
  google_doc_url
}
```

2.3 HHH Evaluation & Confidence Scoring Architecture



Agent Outputs → Final Synthesizer (GPT-4o)



$$(H + H + H) / 3$$



Example Output:

Market Data:

$$[H:85 \mid H:90 \mid H:80] = 85\%$$

Market Intel:

$$[H:80 \mid H:85 \mid H:75] = 80\%$$

Strategic:

$$[H:90 \mid H:85 \mid H:88] = 87.67\%$$

Risk:

$$[H:75 \mid H:90 \mid H:80] = 81.67\%$$

Competitive:

$$[H:85 \mid H:80 \mid H:85] = 83.33\%$$



Step 1: Calculate Base

Weighted Formula:

$$\begin{aligned} & (85 \times 0.25) + \\ & (80 \times 0.25) + \\ & (87.67 \times 0.20) + \\ & (81.67 \times 0.15) + \\ & (83.33 \times 0.15) \end{aligned}$$

$$= 21.25 + 20.00 + 17.53 + 12.25 + 12.50$$

$$\text{Base Score} = 83.53\%$$



Step 2: Select Adjustment

Agent evaluates:

- Cross-agent agreement?
- Conflicts detected?
- Missing critical data?

Options:

- [+8%] Strong agreement
- [0%] Moderate agreement
- [-7%] Major conflicts/gaps

Selected: +8%

Reason: "Strong consensus on NVIDIA supply risk"



Step 3: Calculate Final
(Done by JavaScript)

Final = Base + Adjustment

Final = 83.53 + 8.00

Final = 91.53%

This is the "Confidence Score" used throughout the system



Confidence Interpretation

91.53% = HIGH

Ranges:

- > 75%: High (strong)
- 60-75%: Moderate (good)
- 40-60%: Fair (caution)
- < 40%: Low (risky)

3. Component Specifications

3.1 Agent Specifications

Market Data Analysis Agent

yaml

Name: Market Data Analysis Agent

Model: GPT-4o

Temperature: 0.6

Max Tokens: 1500

Weight: 25%

Purpose:

Analyze quantitative market signals and provide data-driven insights on GPU/chip demand, pricing trends, and supply constraints.

Input Schema:

- **strategic_question:** string (required)
- **user_ai_chip_rate:** integer (0-100)
- **user_crypto_rate:** integer (0-100)
- **user_ev_rate:** integer (0-100)
- **user_supply_risk:** integer (0-100)
- **market_intelligence:** string (optional)
- **pdf_text:** string (optional)

Output Schema:

sections:

- Market Signal Analysis
- Key Data Points (3-5 quantitative findings)
- Market Outlook (short-term, medium-term)
- Confidence Assessment

Focus Areas:

- AI chip demand trends (H100, MI300X, Gaudi)
- Crypto mining hardware cycles
- EV component semiconductor requirements
- Supply chain bottlenecks and lead times
- Pricing dynamics (spot vs contract)
- NVIDIA/AMD/Intel supply positioning

Evaluation Criteria (for HHH):

Helpful:

- Provides specific quantitative data
- Actionable market insights

- Clear trend analysis

Harmless:

- Accurate data representation
- No unfounded speculation
- Proper uncertainty disclaimers

Honest:

- Acknowledges data limitations
- Flags low-confidence predictions
- Transparent about sources

Market Intelligence Agent

yaml

Name: Market Intelligence Agent

Model: GPT-4o

Temperature: 0.7

Max Tokens: 1500

Weight: 25%

Purpose:

Analyze competitive landscape, customer behavior, and market positioning for GPU/chip suppliers.

Input Schema:

- **strategic_question:** string (required)
- **company_name:** string
- **job_function:** string
- **competitive_landscape:** string (optional)
- **market_intelligence:** string (optional)
- **pdf_text:** string (optional)

Output Schema:

sections:

- Competitive Landscape
- Strategic Intelligence
- Market Positioning
- Intelligence Quality Assessment

Focus Areas:

- NVIDIA vs AMD vs Intel competitive positioning
- Customer allocation strategies (AWS/Azure/GCP)
- Emerging competitors and market entrants
- CUDA lock-in dynamics and switching costs
- Partnership and acquisition trends

- Channel partner dynamics

Evaluation Criteria (for HHH):

Helpful:

- Specific competitive insights
- Customer behavior patterns
- Actionable positioning recommendations

Harmless:

- Fact-based analysis
- No unverified rumors
- Balanced perspective

Honest:

- Acknowledges intelligence gaps
- Flags source reliability
- Transparent about assumptions

Strategic Analysis Agent

yaml

Name: Strategic Analysis Agent

Model: GPT-4o

Temperature: 0.75

Max Tokens: 1500

Weight: 20%

Purpose:

Evaluate strategic options, model scenarios, and provide forward-looking analysis for semiconductor decisions.

Input Schema:

- **strategic_question:** string (required)
- **strategic_context:** string (optional)
- **organizational_insights:** string (optional)
- **user_ai_chip_rate:** integer
- **user_supply_risk:** integer
- **pdf_text:** string (optional)

Output Schema:

sections:

- Strategic Options Analysis (2-3 options)
- Scenario Planning (best/base/worst case)
- Long-term Implications
- Strategic Recommendations

Focus Areas:

- Strategic option evaluation (buy/build/partner)
- Scenario planning with probabilities
- Long-term market trajectory (12-24 months)
- Technology transition risks
- Organizational capability assessment
- Strategic trade-offs and opportunity costs

Evaluation Criteria (for HHH):

Helpful:

- Clear strategic options
- Well-reasoned scenarios
- Actionable recommendations

Harmless:

- Realistic assumptions
- No over-optimistic projections
- Balanced risk assessment

Honest:

- Explicit assumptions listed
- Uncertainty ranges provided
- Acknowledges scenario limitations

Risk Management Agent

yaml

Name: Risk Management Agent

Model: GPT-4o

Temperature: 0.65

Max Tokens: 1500

Weight: 15%

Purpose:

Identify, assess, and quantify risks across supply chain, geopolitical, technology, and execution dimensions.

Input Schema:

- **strategic_question**: string (required)
- **risk_factors**: string (optional)
- **user_supply_risk**: integer
- **user_regulatory**: integer
- **strategic_context**: string (optional)
- **pdf_text**: string (optional)

Output Schema:

sections:

- Risk Assessment Matrix (Critical/Major/Moderate)
- Supply Chain Risk Analysis
- Geopolitical Risk Assessment
- Mitigation Strategies
- Risk Quantification

Focus Areas:

- Supply chain vulnerabilities (single-source dependencies)
- Geopolitical risks (Taiwan conflict, export controls)
- Technology risks (yield issues, obsolescence)
- Execution risks (capability, timeline, budget)
- Financial risks (cost overruns, ROI uncertainty)
- Reputational risks (customer satisfaction)

Evaluation Criteria (for HHH):

Helpful:

- Quantified risk impacts
- Specific mitigation strategies
- Actionable risk reduction steps

Harmless:

- Realistic risk assessments
- No fear-mongering
- Balanced severity ratings

Honest:

- Acknowledges assessment limitations
- Flags data gaps
- Transparent about confidence levels

Competitive Analysis Agent

yaml

Name: Competitive Analysis Agent

Model: GPT-4o

Temperature: 0.7

Max Tokens: 1500

Weight: 15%

Purpose:

Analyze competitor strategies, market positioning, and competitive responses for semiconductor decisions.

Input Schema:

- **strategic_question:** string (required)

- **competitive_landscape**: string (optional)
- **company_name**: string
- **job_function**: string
- **market_intelligence**: string (optional)
- **pdf_text**: string (optional)

Output Schema:

sections:

- Competitor Analysis (NVIDIA/AMD/Intel)
- Competitive Positioning
- Competitive Threats
- Competitive Response Analysis
- Strategic Implications

Focus Areas:

- Competitor capability assessment
- Competitive strategy patterns
- Customer competitive dynamics (AWS vs Azure vs GCP)
- Technology positioning (CUDA vs ROCm, ASICs)
- Market share trends
- Competitive response scenarios

Evaluation Criteria (for HHH):

Helpful:

- Specific competitor insights
- Response scenario modeling
- Timing advantage identification

Harmless:

- Fact-based competitor analysis
- No unsubstantiated claims
- Balanced competitive assessment

Honest:

- Acknowledges intelligence gaps
- Flags source reliability
- Transparent about competitive unknowns

Final Synthesizer

yaml

Name: Final Synthesizer

Model: GPT-4o

Temperature: 0.7

Max Tokens: 4000

Weight: N/A (evaluator, not evaluated)

Purpose:

Integrate 5 specialized agent analyses using HHH framework, cross-validate findings, and generate executive report with confidence scoring.

Input Schema:

- **strategic_question**: string (required)
- **all_user_context**: object (complete form data)
- **agent_outputs**: array[5] (all agent responses)

Output Schema:

sections:

- Executive Summary
- Integrated Analysis Summary
- Detailed Market Comparison Table
- Cross-Validation Table
- Strategic Insights
- Top 3 Decision Factors
- Strategic Recommendations (30-day, 90-day)
- Risk/Opportunity Matrix
- Executive Decision Framework
- Limitations & Follow-Up
- Agent Reliability Assessment (HHH scores)
- Confidence Score Calculation (3 steps)

Process:

1. Evaluate each agent on HHH (0-100 scale)
2. **Calculate agent average**: $(H+H+H)/3$
3. Cross-validate findings across agents
4. Identify reinforcing patterns and conflicts
5. Select adjustment (+8%, 0%, -7%)
6. Generate comprehensive executive report
7. Output HHH scores in exact format for parsing

Critical Instructions:

- **Use exact HHH format**: [Helpful: X | Harmless: X | Honest: X] = Avg: X.XX%
- Select ONE adjustment with clear reasoning
- Do NOT perform final calculation (JavaScript handles this)
- Maintain executive brevity while thorough
- Prioritize actionable insights over theory

3.2 JavaScript Confidence Calculator

javascript

```
/**
 * Confidence Score Calculator
 *
 * Purpose: Extract HHH scores from synthesizer output and calculate
 *          final confidence score using weighted formula.
 *
 * Input: Synthesizer text output with embedded HHH scores
 * Output: Numerical confidence score (0-100, 2 decimals)
 *
 * Algorithm:
 * 1. Parse HHH scores for 5 agents using regex
 * 2. Calculate weighted base score
 * 3. Detect adjustment from synthesizer text
 * 4. Compute final: base + adjustment
 * 5. Return formatted score
 */

const calculateConfidenceScore = () => {
  if (!synthesizedReport) return "";

  // Extract HHH average scores for each agent
  const extractHHH = (agentName) => {
    // Pattern: Agent Name: [Helpful: X | Harmless: X | Honest: X] = Avg: XX.XX%
    const pattern = new RegExp(
      agentName +
      '\s*\[Helpful:\s*(\d+)\s*\| Harmless:\s*(\d+)\s*\| Honest:\s*(\d+)\s*\] = Avg:\s*(\d+\.\d+)%' +
      '\s*'
    );

    const match = synthesizedReport.match(pattern);

    if (match && match[4]) {
      return parseFloat(match[4]);
    }
    return 0;
  };

  // Extract all 5 agent scores
  const scores = {
    marketData: extractHHH('Market Data Analysis'),
```

```

    marketIntel: extractHHH('Market Intelligence'),
    strategic: extractHHH('Strategic Analysis'),
    risk: extractHHH('Risk Management'),
    competitive: extractHHH('Competitive Analysis')
  };

  console.log('Extracted HHH scores:', scores);

  // Calculate base score using weighted formula
  // Weights: MD(25%) + MI(25%) + SA(20%) + RM(15%) + CA(15%) = 100%
  const baseScore =
    (scores.marketData * 0.25) +
    (scores.marketIntel * 0.25) +
    (scores.strategic * 0.20) +
    (scores.risk * 0.15) +
    (scores.competitive * 0.15);

  console.log('Base confidence score:', baseScore);

  // Extract adjustment from synthesizer's Step 2
  let adjustment = 0;

  if (synthesizedReport.match(/Selected adjustment:\s*\+8%/i) ||
    synthesizedReport.match(/Strong.*?agreement.*?\+8%/i)) {
    adjustment = 8;
  } else if (synthesizedReport.match(/Selected adjustment:\s*\-7%/i) ||
    synthesizedReport.match(/Major conflicts.*?\-7%/i)) {
    adjustment = -7;
  } else if (synthesizedReport.match(/Selected adjustment:\s*0%/i) ||
    synthesizedReport.match(/Moderate agreement.*?0%/i)) {
    adjustment = 0;
  }

  console.log('Applied adjustment:', adjustment);

  // Calculate final confidence score
  const finalScore = baseScore + adjustment;

  console.log('Final confidence score:', finalScore);

  // Validate result
  if (isNaN(finalScore)) {
    console.error('Confidence calculation failed - returning empty string');
    return '';
  }

```

```

}

// Return formatted score (2 decimal places)
return finalScore.toFixed(2);
};

// Usage in Extract & Validate node
return {
  json: {
    // ... other fields ...
    confidence_score: calculateConfidenceScore(),
    // ... other fields ...
  }
};

```

3.3 PDF Parser Specification

javascript

```

/**
 * PDF Parser Module
 *
 * Purpose: Extract text from uploaded PDF documents for context enrichment
 * Library: pdf-parse (via N8N code node)
 *
 * Input: Base64-encoded PDF or Buffer
 * Output: Extracted text, page count, metadata
 */

const parsePDF = async (pdfData) => {
  if (!pdfData) {
    return { pdf_text: '', pdf_error: 'No PDF provided' };
  }

  // Decode base64 if needed
  let pdfBuffer;
  if (typeof pdfData === 'string') {
    pdfBuffer = Buffer.from(pdfData, 'base64');
  } else {
    pdfBuffer = pdfData;
  }

  // Parse PDF using pdf-parse
  import pdfParse from 'pdf-parse';

```

```

try {
  const data = await pdfParse(pdfBuffer);

  return {
    pdf_text: data.text,
    pdf_pages: data.numpages,
    pdf_metadata: {
      title: data.info?.Title || 'Unknown',
      author: data.info?.Author || 'Unknown',
      subject: data.info?.Subject || '',
      creator: data.info?.Creator || '',
      producer: data.info?.Producer || '',
      creation_date: data.info?.CreationDate || '',
      mod_date: data.info?.ModDate || ''
    },
    pdf_processed_at: new Date().toISOString(),
    pdf_size_bytes: pdfBuffer.length,
    pdf_char_count: data.text.length
  };
} catch (error) {
  console.error('PDF parsing error:', error);
  return {
    pdf_text: '',
    pdf_error: error.message,
    pdf_processing_failed: true
  };
}
};

// Maximum PDF size: 10MB
const MAX_PDF_SIZE = 10 * 1024 * 1024;

// Validation
if (pdfBuffer.length > MAX_PDF_SIZE) {
  throw new Error('PDF file exceeds maximum size of 10MB');
}

```

4. Data Flow

4.1 Request Flow

1. User Submission

- └ HTTP POST /webhook/stratai-submit
- └ Headers: Content-Type: application/json
- └ Body: {
 - user_email: string,
 - company_name: string,
 - job_function: string,
 - strategic_question: string (required, 10-500 chars),
 - market_intelligence: string (optional, max 5000 chars),
 - strategic_context: string (optional, max 5000 chars),
 - risk_factors: string (optional, max 5000 chars),
 - competitive_landscape: string (optional, max 5000 chars),
 - organizational_insights: string (optional, max 5000 chars),
 - user_ai_chip_rate: integer (0-100),
 - user_crypto_rate: integer (0-100),
 - user_ev_rate: integer (0-100),
 - user_supply_risk: integer (0-100),
 - user_regulatory: integer (0-100),
 - user_consumer_rate: integer (0-100),
 - pdf_file: base64_string (optional, max 10MB)}

2. Webhook Trigger (N8N)

- └ Receives POST request
- └ Validates webhook signature
- └ Passes to next node

3. Extract Form Data (Code Node)

- └ Validates required fields
- └ Sanitizes inputs
- └ Generates session_id
- └ Parses integers from strings
- └ Outputs: Clean JSON object

4. PDF Processing (Conditional)

- └ IF pdf_file exists
 - └ Decode base64
 - └ Parse with pdf-parse
 - └ Extract text & metadata
 - └ Merge with form data
- └ ELSE: Skip to agent execution

5. Multi-Agent Execution (Parallel)

- └ Trigger 5 OpenAI API calls simultaneously
- └ Each agent receives:

- | — System prompt (agent-specific)
- | — User message (with all context)
- | — Temperature & token limits
- Wait for all responses
- Outputs: Array[5] of agent analyses

6. Merge Agents (N8N Node)

- Combines 5 agent outputs
- Preserves order
- Passes to synthesizer

7. Final Synthesizer (GPT-4o)

- Receives all 5 agent outputs
- Evaluates each on HHH framework
- Cross-validates findings
- Selects confidence adjustment
- Generates comprehensive report
- Outputs: Markdown report with HHH scores

8. Extract & Validate (Code Node)

- Parses synthesizer output
- Extracts HHH scores (regex)
- Calculates confidence score
 - | — Base = weighted average
 - | — Adjustment = detected from text
 - | — Final = base + adjustment
- Extracts report sections
- Outputs: Structured JSON

9. Parallel Output Generation

- Create Google Doc
 - | — Format report as Markdown
 - | — Generate title with timestamp
 - | — Call Google Docs API
 - | — Returns: documentUrl
- Save to Airtable
 - | — Append to Reports table
 - | — Store all metadata
 - | — Link session_id

10. Response (Webhook Response Node)

- Returns to user: {
 success: true,
 report_id: "RPT-1234567890",

```
confidence_score: "85.42",
google_doc_url: "https://docs.google.com/...",
timestamp: "2025-01-18T..."
}
```

Total Processing Time: 30-60 seconds

4.2 Data Schema

Input Schema (Form Submission)

```
typescript
interface StratAIRequest {
  // User Information
  user_email: string;      // Email format, required
  company_name: string;    // 1-200 chars, required
  job_function: string;    // 1-200 chars, required

  // Strategic Question
  strategic_question: string; // 10-500 chars, required

  // Context Fields (all optional, max 5000 chars each)
  market_intelligence?: string;
  strategic_context?: string;
  risk_factors?: string;
  competitive_landscape?: string;
  organizational_insights?: string;

  // Market Signals (0-100 scale)
  user_ai_chip_rate: number; // Integer 0-100, required
  user_crypto_rate: number;  // Integer 0-100, required
  user_ev_rate: number;      // Integer 0-100, required
  user_supply_risk: number;  // Integer 0-100, required
  user_regulatory: number;   // Integer 0-100, required
  user_consumer_rate: number; // Integer 0-100, required

  // Optional PDF Upload
  pdf_file?: string; // Base64-encoded, max 10MB
}
```

Intermediate Schema (After Form Extraction)

```
typescript
interface ProcessedInput extends StratAIRequest {
  timestamp: string; // ISO 8601 format
}
```

```

    session_id: string;           // "SESSION-{timestamp}-{random}"
    pdf_processed: boolean;       // True if PDF was uploaded
    pdf_text?: string;           // Extracted text from PDF
    pdf_pages?: number;          // Number of pages
    pdf_metadata?: PDFMetadata;   // PDF document metadata
  }

```

```

interface PDFMetadata {
  title: string;
  author: string;
  subject: string;
  creator: string;
  producer: string;
  creation_date: string;
  mod_date: string;
}

```

Agent Output Schema

typescript

```

interface AgentOutput {
  role: string;           // "assistant"
  content: string;        // Agent's analysis (Markdown)
  refusal: null;
  annotations: any[];
}

```

```

interface AgentResponse {
  index: number;
  message: AgentOutput;
  logprobs: null;
  finish_reason: string;   // "stop"
}

```

Synthesizer Output Schema

typescript

```

interface SynthesizerOutput {
  choices: [{
    message: {
      role: "assistant";
      content: string;    // Full report in Markdown
    }
  }]
}

```

```
}
```

Final Output Schema (Extract & Validate)

typescript

```
interface StratAIReport {  
  // Metadata  
  report_id: string;      // "RPT-{timestamp}"  
  timestamp: string;      // ISO 8601  
  session_id: string;     // Matching input  
  
  // User Info  
  user_email: string;  
  company_name: string;  
  job_function: string;  
  strategic_question: string;  
  
  // Calculated Metrics  
  confidence_score: string; // "XX.XX" format (2 decimals)  
  
  // PDF Info  
  pdf_included: boolean;  
  
  // Full Report  
  full_report: string;     // Complete Markdown report  
  
  // Extracted Sections  
  executive_summary: string;  
  market_comparison: string;  
  cross_validation: string;  
  strategic_insights: string;  
  decision_factors: string;  
  recommendations: string;  
  risk_matrix: string;  
  executive_framework: string;  
  limitations: string;  
  reliability: string;  
  
  // Market Signals (copied from input)  
  ai_chip_rate: number;  
  crypto_rate: number;  
  ev_rate: number;  
  supply_risk: number;  
  regulatory: number;  
  consumer_rate: number;
```

```
}
```

Airtable Storage Schema

typescript

// Reports Table

```
interface AirtableReport {
```

// Primary Key

```
report_id: string;           // "RPT-{timestamp}"
```

// Timestamps

```
timestamp: Date;
```

// User Info

```
user_email: string;
```

```
company_name: string;
```

```
job_function: string;
```

// Question

```
strategic_question: string; // Long text
```

// Metrics

```
confidence_score: number;    // Decimal (2 places)
```

// Market Signals

```
ai_chip_rate: number;
```

```
crypto_rate: number;
```

```
ev_rate: number;
```

```
supply_risk: number;
```

```
regulatory: number;
```

```
consumer_rate: number;
```

// Session

```
session_id: string;
```

// Google Doc

```
google_doc_url: string;      // URL field
```

// Report Sections (Long text fields)

```
executive_summary: string;
```

```
recommendations: string;
```

```
limitations: string;
```

```
}
```

// Feedback Table

```

interface AirtableFeedback {
  feedback_id: string;      // "FB-{timestamp}"
  report_id: string;        // Link to Reports
  rating: number;           // 1-5
  feedback_text: string;    // Long text
  useful: boolean;
  would_recommend: boolean;
  user_email: string;
  timestamp: Date;
}

// Alerts Table
interface AirtableAlert {
  alert_id: string;         // "ALERT-{timestamp}"
  report_id: string;        // Link to Reports
  alert_type: string;       // "low_confidence" | "competitor" | "system"
  confidence_score: number;
  user_email: string;
  timestamp: Date;
  action_taken: string;
}

// Users Table
interface AirtableUser {
  user_email: string;       // Primary key
  user_name: string;
  company_name: string;
  job_function: string;
  signup_timestamp: Date;
  onboarding_stage: string; // "welcome" | "active" | "churned"
  reports_generated: number;
  last_report_date: Date;
}

```

5. API Documentation

5.1 REST API Endpoints

Main Analysis Endpoint

http

POST /webhook/stratai-submit

Host: your-n8n-instance.com

Content-Type: application/json

Request Body:

```
{
  "user_email": "director@azure.com",
  "company_name": "Microsoft Azure",
  "job_function": "Director of GPU Allocation",
  "strategic_question": "Should we implement tiered rate-limiting or uniform throttling for GPU capacity?",
  "market_intelligence": "Managing 85 Azure OpenAI accounts generating $45M ARR. Current challenge: 10 enterprise customers consume 68% of GPU allocation...",
  "strategic_context": "Team goal: maintain 92% customer satisfaction while protecting GPU capacity for growth accounts...",
  "risk_factors": "Top customer dependency: Salesforce alone is $8M ARR with contract renewal in 90 days...",
  "competitive_landscape": "AWS Bedrock offering tiered capacity guarantees. OpenAI direct API customers get priority...",
  "organizational_insights": "My team: 12 customer success engineers, 8 on enterprise accounts, 4 on SMB segment...",
  "user_ai_chip_rate": 88,
  "user_crypto_rate": 12,
  "user_ev_rate": 20,
  "user_supply_risk": 65,
  "user_regulatory": 55,
  "user_consumer_rate": 48
}
```

Response (200 OK):

```
{
  "success": true,
  "report_id": "RPT-1737158400123",
  "confidence_score": "72.45",
  "google_doc_url": "https://docs.google.com/document/d/1abc...xyz/edit",
  "timestamp": "2025-01-18T10:30:45.123Z",
  "message": "Strategic analysis complete. Report generated successfully."
}
```

Response (400 Bad Request):

```
{
  "success": false,
  "error": "Validation failed",
  "details": {
    "strategic_question": "Field is required and must be 10-500 characters"
  }
}
```

Response (500 Internal Server Error):

```
{
  "success": false,
  "error": "Analysis generation failed",
  "message": "OpenAI API rate limit exceeded. Please try again in 60 seconds."
}
```

Chat Assistant Endpoint

http

POST /webhook/stratai-chat

Host: your-n8n-instance.com

Content-Type: application/json

Request Body:

```
{
  "session_id": "SESSION-1737158400123-abc123",
  "report_id": "RPT-1737158400123",
  "message": "Can you explain why the confidence score is 72%?",
  "user_email": "director@azure.com"
}
```

Response (200 OK):

```
{
  "response": "The 72.45% confidence score reflects moderate agreement among our 5 AI agents with some conflicting views on customer retention strategies. Here's the breakdown:\n\n**Base Score: 74.45%**\n- Market Data Agent (85% HHH): Strong quantitative analysis\n- Market Intelligence (78% HHH): Good competitive insights\n- Strategic Analysis (82% HHH): Solid scenario planning\n- Risk Management (68% HHH): Some uncertainty on churn predictions\n- Competitive Analysis (75% HHH): Good positioning analysis\n\n**Adjustment: -2% (Minor conflicts)**\n\nThe Risk Management agent flagged uncertainty about how customers would react to rate limiting, while Strategic Analysis was more confident in the tiered approach.\n\n**What this means:**\n\nA 72% confidence score is \"Moderate-High\" - the recommendation is solid but acknowledge there's some uncertainty, particularly around customer behavior predictions. I'd suggest monitoring customer feedback closely in the first 30 days of any changes.",
  "timestamp": "2025-01-18T10:35:12.456Z",
  "session_id": "SESSION-1737158400123-abc123"
}
```

Feedback Submission Endpoint

http

POST /webhook/stratai-feedback

Host: your-n8n-instance.com
Content-Type: application/json

Request Body:

```
{  
  "report_id": "RPT-1737158400123",  
  "rating": 5,  
  "feedback_text": "Excellent analysis - helped us make a confident decision on tiered  
rate-limiting. The confidence scoring transparency was particularly valuable.",  
  "useful": true,  
  "would_recommend": true,  
  "user_email": "director@azure.com"  
}
```

Response (200 OK):

```
{  
  "success": true,  
  "feedback_id": "FB-1737160000789",  
  "message": "Thank you for your feedback!"  
}
```

Analytics Dashboard Endpoint

http

GET /webhook/stratai-analytics

Host: your-n8n-instance.com

Response (200 OK):

```
{  
  "total_reports": 247,  
  "avg_confidence_score": "76.84",  
  "total_feedback": 89,  
  "avg_rating": "4.6",  
  "reports_by_company": {  
    "Microsoft Azure": 45,  
    "Google Cloud": 38,  
    "Oracle Cloud": 22,  
    "NVIDIA": 18,  
    "Tesla": 12,  
    "Others": 112  
  },  
  "recent_reports": [  
    {  
      "report_id": "RPT-1737158400123",
```

```
    "company": "Microsoft Azure",
    "confidence": "72.45",
    "timestamp": "2025-01-18T10:30:45.123Z"
  },
  // ... 9 more recent reports
],
"generated_at": "2025-01-18T12:00:00.000Z"
}
```

5.2 Webhook Events

Email Notification Trigger

http

POST /webhook/stratai-email-trigger

Content-Type: application/json

```
{
  "report_id": "RPT-1737158400123",
  "user_email": "director@azure.com",
  "company_name": "Microsoft Azure",
  "confidence_score": "72.45",
  "google_doc_url": "https://docs.google.com/...",
  "strategic_question": "Should we implement tiered rate-limiting?",
  "notification_type": "report_complete"
}
```

Slack Notification Trigger

http

POST /webhook/stratai-slack-notify

Content-Type: application/json

```
{
  "report_id": "RPT-1737158400123",
  "company_name": "Microsoft Azure",
  "confidence_score": "72.45",
  "strategic_question": "Should we implement tiered rate-limiting?",
  "google_doc_url": "https://docs.google.com/...",
  "slack_channel": "#strategy-team"
}
```

Low Confidence Alert Trigger

http

POST /webhook/stratai-confidence-check

Content-Type: application/json

```
{
  "report_id": "RPT-1737158400123",
  "confidence_score": 55.23,
  "user_email": "director@azure.com",
  "company_name": "Microsoft Azure",
  "strategic_question": "Should we invest in custom ASIC development?",
  "limitations": "Missing critical data on ASIC development costs. Major conflicts between agents on market timing."
}
```

5.3 Rate Limits

yaml

Global Limits:

- 100 requests per hour per IP
- 1000 requests per day per IP

Endpoint-Specific Limits:

/stratai-submit:

- 10 requests per hour per user_email
- Reason: Each analysis consumes ~\$0.30 in API costs

/stratai-chat:

- 50 requests per hour per session_id
- Reason: Prevent chat spam

/stratai-feedback:

- 5 requests per hour per user_email
- Reason: Prevent feedback spam

/stratai-analytics:

- 100 requests per hour (public endpoint)
- Reason: Dashboard queries

Error Response (429 Too Many Requests):

```
{
  "success": false,
  "error": "Rate limit exceeded",
  "retry_after": 3600,
  "limit": "10 requests per hour",
  "current_usage": 10
}
```

}

6. Security & Compliance

6.1 Authentication & Authorization

yaml

Authentication Method: API Key + Webhook Signatures

Webhook Security:

- N8N webhook URLs contain random UUID
- **Example:** /webhook/stratai-submit-a7b3c9d2-4e5f-6a7b-8c9d-0e1f2a3b4c5d
- URLs are secret and should not be exposed publicly

API Key Requirements (if implementing):

- **Header:** X-StratAI-API-Key
- **Format:** stratai_key_32_char_alphanumeric
- **Rotation:** Every 90 days
- **Storage:** Environment variables, never in code

User Authentication:

- **Currently:** Email-based (honor system)
- **Recommended upgrade:** OAuth 2.0 with Google/Microsoft
- **Session management:** JWT tokens with 24-hour expiry

Authorization Levels:

- **User:** Can create reports, view own reports
- **Admin:** Can view all reports, access analytics
- **System:** Internal webhook-to-webhook communication

6.2 Data Privacy & GDPR Compliance

yaml

Data Classification:

Personal Data (PII):

- user_email (email address)
- user_name (if collected)
- company_name (potentially identifying)

Sensitive Business Data:

- strategic_question
- market_intelligence

- strategic_context
- risk_factors
- competitive_landscape
- organizational_insights
- Generated reports

Non-Sensitive:

- Market signal rates (numeric values)
- Confidence scores
- Timestamps

Data Retention:

Airtable (Primary Storage):

- Reports: 24 months, then archived
- Feedback: Indefinite (anonymized after 12 months)
- User data: Active + 12 months after last activity

Google Docs:

- User-controlled (we generate, user owns)
- Not automatically deleted by StratAI

N8N Logs:

- Execution logs: 7 days
- Error logs: 30 days

LLM Provider Logs:

- OpenAI: 30 days (per their policy)
- Anthropic: 90 days (per their policy)

GDPR Rights Implementation:

Right to Access:

- Endpoint: GET /api/user/{email}/data
- Returns: All reports, feedback, metadata

Right to Deletion:

- Endpoint: DELETE /api/user/{email}
- Actions:
 1. Delete from Airtable (Reports, Feedback, Users)
 2. Email user with Google Doc URLs for manual deletion
 3. Purge from N8N logs
 4. Note: Cannot delete from LLM provider logs

Right to Portability:

- Format: JSON export of all user data

- **Includes:** All report JSONs + Google Doc links

Right to Rectification:

- Users can edit reports via Google Docs directly
- **Metadata corrections:** Contact admin

Data Processing Agreements (DPA):

Required with:

- OpenAI (Sub-processor)
- Anthropic (Sub-processor)
- Google Cloud (Sub-processor)
- Airtable (Sub-processor)
- N8N Cloud (if using hosted version)

Consent Management:

- Explicit consent collected during signup
- **Purpose:** Strategic analysis generation
- **Lawful basis:** Legitimate interest (B2B service)
- **Opt-out:** User can request deletion anytime

6.3 Data Encryption

yaml

Encryption in Transit:

All APIs:

- TLS 1.3
- **Certificate:** Let's Encrypt or commercial CA
- HTTPS only, HTTP redirects to HTTPS

Webhook Communications:

- **N8N to OpenAI:** HTTPS (TLS 1.3)
- **N8N to Anthropic:** HTTPS (TLS 1.3)
- **N8N to Google APIs:** HTTPS (TLS 1.3)
- **N8N to Airtable:** HTTPS (TLS 1.3)

Email:

- SMTP with STARTTLS
- SPF, DKIM, DMARC configured

Encryption at Rest:

Airtable:

- AES-256 encryption (managed by Airtable)
- Backups encrypted

Google Drive/Docs:

- AES-256 encryption (managed by Google)

N8N Database:

- **Self-hosted**: Configure disk encryption (LUKS)
- **Cloud**: Managed encryption by provider

Environment Variables:

- Encrypted with deployment platform secret management
- Never stored in plain text in repositories

API Keys Storage:

- **N8N Credentials**: Encrypted in N8N database with master key
- **Master key**: Stored in environment variable or secret manager
- **Rotation**: Automated every 90 days (recommended)

6.4 Input Validation & Sanitization

javascript

```
/**
```

```
 * Input Validation Rules
```

```
 */
```

```
const validationRules = {
```

```
  user_email: {
    type: 'email',
    required: true,
    maxLength: 255,
    pattern: /^[^\\s@]+@[^\\s@]+\\.\\.[^\\s@]+$/
  },
```

```
  company_name: {
    type: 'string',
    required: true,
    minLength: 1,
    maxLength: 200,
    sanitize: {
      stripHtml: true,
      trim: true
    }
  },
```

```
  strategic_question: {
    type: 'string',
```

```

    required: true,
    minLength: 10,
    maxLength: 500,
    sanitize: {
      stripHtml: true,
      trim: true,
      preventInjection: true
    }
  },

  market_intelligence: {
    type: 'string',
    required: false,
    maxLength: 5000,
    sanitize: {
      stripHtml: true,
      trim: true,
      preventInjection: true
    }
  },

  user_ai_chip_rate: {
    type: 'integer',
    required: true,
    min: 0,
    max: 100
  },

  pdf_file: {
    type: 'base64',
    required: false,
    maxSize: 10485760, // 10MB in bytes
    allowedTypes: ['application/pdf'],
    validate: (data) => {
      // Verify it's actually a PDF
      const buffer = Buffer.from(data, 'base64');
      return buffer.toString('utf8', 0, 4) === '%PDF';
    }
  }
};

/**
 * Sanitization Functions
 */

```

```

const sanitize = {
  stripHtml: (input) => {
    return input.replace(/<[>]*>/g, "");
  },

  preventInjection: (input) => {
    // Escape special characters that could be used in injection attacks
    const escapeChars = {
      '<': '&lt;',
      '>': '&gt;',
      '"': '&quot;',
      "'": '&#x27;',
      '/': '&#x2F;',
    };
    return input.replace(/<>"'/g, (char) => escapeChars[char]);
  },

  trim: (input) => {
    return input.trim();
  }
};

/**
 * Validation Implementation
 */

const validateInput = (data, rules) => {
  const errors = {};

  for (const [field, rule] of Object.entries(rules)) {
    const value = data[field];

    // Required check
    if (rule.required && !value) {
      errors[field] = `${field} is required`;
      continue;
    }

    // Skip validation if optional and not provided
    if (!rule.required && !value) continue;

    // Type validation
    if (rule.type === 'email' && !rule.pattern.test(value)) {

```

```

    errors[field] = `${field} must be a valid email`;
  }

  if (rule.type === 'integer') {
    const num = parseInt(value);
    if (isNaN(num) || num < rule.min || num > rule.max) {
      errors[field] = `${field} must be between ${rule.min} and ${rule.max}`;
    }
  }

  // Length validation
  if (rule.minLength && value.length < rule.minLength) {
    errors[field] = `${field} must be at least ${rule.minLength} characters`;
  }

  if (rule.maxLength && value.length > rule.maxLength) {
    errors[field] = `${field} must be less than ${rule.maxLength} characters`;
  }

  // Custom validation
  if (rule.validate && !rule.validate(value)) {
    errors[field] = `${field} failed custom validation`;
  }
}

return {
  isValid: Object.keys(errors).length === 0,
  errors
};
};

```

6.5 Security Best Practices

yaml

API Security:

- Rate limiting on all endpoints
- CORS properly configured
- No sensitive data in URLs (use POST body)
- Request size limits (max 10MB)
- Timeouts on all external API calls (30s)

Code Security:

- No hardcoded credentials
- Environment variables for all secrets

- Regular dependency updates (npm audit)
- Code reviews for all changes
- Principle of least privilege

Infrastructure Security:

- Firewall rules (whitelist approach)
- Regular security patches
- Intrusion detection system (IDS)
- DDoS protection (Cloudflare)
- Backup encryption

Logging & Monitoring:

- Log all API requests (without sensitive data)
- Alert on unusual patterns
- Monitor for SQL injection attempts
- Track failed authentication attempts
- Regular log reviews

Incident Response:

- Documented response plan
- **Contact:** security@stratai.com
- **Response SLA:** 4 hours for critical issues
- Post-mortem required for all incidents

7. Deployment Guide

7.1 Infrastructure Requirements

yaml

Minimum Requirements:

N8N Instance:

CPU: 2 vCPUs

RAM: 4GB

Storage: 20GB SSD

Network: 100 Mbps

Database (if self-hosting):

PostgreSQL 12+

CPU: 1 vCPU

RAM: 2GB

Storage: 10GB SSD

Recommended Production Setup:

N8N Instance:

CPU: 4 vCPUs

RAM: 8GB

Storage: 50GB SSD

Network: 1 Gbps

Load Balancer: Yes

Auto-scaling: 2-5 instances

Database:

PostgreSQL 14+

CPU: 2 vCPUs

RAM: 4GB

Storage: 50GB SSD with automated backups

High Availability: Multi-AZ deployment

Cloud Provider Options:

AWS:

- EC2 t3.medium or t3.large
- RDS PostgreSQL
- Application Load Balancer
- CloudWatch for monitoring
- **Estimated cost:** \$150-300/month

Google Cloud:

- Compute Engine n1-standard-2 or n1-standard-4
- Cloud SQL PostgreSQL
- Cloud Load Balancing
- Cloud Monitoring
- **Estimated cost:** \$140-280/month

Azure:

- Virtual Machines B2s or B2ms
- Azure Database for PostgreSQL
- Azure Load Balancer
- Azure Monitor
- **Estimated cost:** \$160-300/month

DigitalOcean:

- **Droplet:** 4GB or 8GB
- Managed PostgreSQL
- Load Balancer
- Monitoring included
- **Estimated cost:** \$80-160/month

N8N Cloud (Recommended for Quick Start):

- Fully managed N8N instance
- Auto-scaling included
- Built-in monitoring
- SSL certificates managed
- **Estimated cost:** \$50-200/month depending on usage
- **URL:** <https://n8n.cloud>

Storage Requirements:

Airtable:

- **Free tier:** 1,200 records per base (sufficient for MVP)
- **Plus tier:** \$20/user/month (50,000 records, recommended)
- **Pro tier:** \$45/user/month (100,000 records, for scale)

Google Drive:

- **Free tier:** 15GB (approximately 15,000 reports)
- **Google Workspace:** \$6-18/user/month (unlimited storage)

Bandwidth Estimates:

- **Per analysis:** ~2MB upload, ~5MB download
- **1,000 analyses/month:** ~7GB total bandwidth
- Included in most cloud provider base costs

7.2 Installation Steps

Option 1: N8N Cloud (Recommended for Quick Start)

bash

Step 1: Sign up for N8N Cloud

Visit: <https://n8n.cloud>

Create account and select plan

Step 2: Access your N8N instance

You'll receive a URL like: <https://your-org.app.n8n.cloud>

Step 3: Import workflows

1. Navigate to Settings → Import from File

2. Import each workflow JSON file:

- stratai-main-workflow.json

- stratai-chat-assistant.json

- stratai-feedback.json

- stratai-admin-analytics.json

- stratai-email-notifications.json

```
# - stratai-scheduled-reports.json
# - stratai-low-confidence-alert.json
# - stratai-slack-notifications.json
# - stratai-pdf-export.json
# - stratai-teams-notifications.json
# - stratai-monthly-executive-summary.json
# - stratai-competitor-monitoring.json
# - stratai-user-onboarding.json
```

```
# Step 4: Configure credentials (see section 7.3)
```

```
# Step 5: Activate workflows
```

```
# Enable each workflow individually after testing
```

```
# Deployment time: 2-4 hours
```

Option 2: Self-Hosted on Ubuntu 22.04

```
bash
```

```
# Step 1: Provision server
```

```
# AWS EC2, Google Compute, Azure VM, or DigitalOcean Droplet
```

```
# Minimum: 2 vCPU, 4GB RAM, 20GB SSD
```

```
# Step 2: SSH into server
```

```
ssh ubuntu@your-server-ip
```

```
# Step 3: Update system
```

```
sudo apt update && sudo apt upgrade -y
```

```
# Step 4: Install Node.js 18.x
```

```
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt install -y nodejs
```

```
# Verify installation
```

```
node --version # Should show v18.x.x
```

```
npm --version # Should show 9.x.x
```

```
# Step 5: Install N8N globally
```

```
sudo npm install -g n8n
```

```
# Step 6: Install PostgreSQL (optional but recommended)
```

```
sudo apt install -y postgresql postgresql-contrib
```

```
# Create database and user
```

```
sudo -u postgres psql
CREATE DATABASE n8n;
CREATE USER n8n_user WITH PASSWORD 'your_secure_password';
GRANT ALL PRIVILEGES ON DATABASE n8n TO n8n_user;
\q
```

Step 7: Configure N8N environment variables

```
sudo nano /etc/systemd/system/n8n.service
```

Add the following content:

```
[Unit]
```

```
Description=N8N Workflow Automation
```

```
After=network.target
```

```
[Service]
```

```
Type=simple
```

```
User=ubuntu
```

```
Environment="N8N_BASIC_AUTH_ACTIVE=true"
```

```
Environment="N8N_BASIC_AUTH_USER=admin"
```

```
Environment="N8N_BASIC_AUTH_PASSWORD=your_secure_password"
```

```
Environment="N8N_HOST=0.0.0.0"
```

```
Environment="N8N_PORT=5678"
```

```
Environment="N8N_PROTOCOL=https"
```

```
Environment="WEBHOOK_URL=https://your-domain.com"
```

```
Environment="DB_TYPE=postgresdb"
```

```
Environment="DB_POSTGRESDB_HOST=localhost"
```

```
Environment="DB_POSTGRESDB_PORT=5432"
```

```
Environment="DB_POSTGRESDB_DATABASE=n8n"
```

```
Environment="DB_POSTGRESDB_USER=n8n_user"
```

```
Environment="DB_POSTGRESDB_PASSWORD=your_secure_password"
```

```
Environment="EXECUTIONS_DATA_SAVE_ON_SUCCESS=all"
```

```
Environment="EXECUTIONS_DATA_SAVE_ON_ERROR=all"
```

```
Environment="EXECUTIONS_DATA_SAVE_MANUAL_EXECUTIONS=true"
```

```
ExecStart=/usr/bin/n8n
```

```
Restart=always
```

```
RestartSec=10
```

```
[Install]
```

```
WantedBy=multi-user.target
```

Save and exit (Ctrl+X, Y, Enter)

Step 8: Enable and start N8N service

```
sudo systemctl daemon-reload
```

```
sudo systemctl enable n8n
sudo systemctl start n8n
```

Check status

```
sudo systemctl status n8n
```

Step 9: Install and configure Nginx as reverse proxy

```
sudo apt install -y nginx certbot python3-certbot-nginx
```

Configure Nginx

```
sudo nano /etc/nginx/sites-available/n8n
```

Add the following configuration:

```
server {
    listen 80;
    server_name your-domain.com;

    location / {
        proxy_pass http://localhost:5678;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        # Increase timeouts for long-running workflows
        proxy_connect_timeout 300s;
        proxy_send_timeout 300s;
        proxy_read_timeout 300s;
    }

    # Increase client body size for PDF uploads
    client_max_body_size 10M;
}
```

Save and exit

Enable site

```
sudo ln -s /etc/nginx/sites-available/n8n /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl restart nginx
```

Step 10: Obtain SSL certificate

```
sudo certbot --nginx -d your-domain.com
```

Follow prompts to obtain certificate

Certbot will automatically configure HTTPS

Step 11: Configure firewall

```
sudo ufw allow 'Nginx Full'
```

```
sudo ufw allow OpenSSH
```

```
sudo ufw enable
```

Step 12: Access N8N

Open browser: https://your-domain.com

Login with credentials from Step 7

Step 13: Import workflows

Follow same import process as N8N Cloud option

Deployment time: 3-6 hours

Option 3: Docker Deployment

bash

Step 1: Install Docker and Docker Compose

```
curl -fsSL https://get.docker.com -o get-docker.sh
```

```
sudo sh get-docker.sh
```

```
sudo usermod -aG docker $USER
```

```
sudo apt install -y docker-compose
```

Log out and back in for group changes to take effect

Step 2: Create project directory

```
mkdir ~/stratai-n8n
```

```
cd ~/stratai-n8n
```

Step 3: Create docker-compose.yml

```
cat > docker-compose.yml << 'EOF'
```

```
version: '3.8'
```

```
services:
```

```
  postgres:
```

```
    image: postgres:14-alpine
```

```
    restart: always
```

environment:

POSTGRES_USER: n8n

POSTGRES_PASSWORD: \${POSTGRES_PASSWORD}

POSTGRES_DB: n8n

volumes:

- postgres-data:/var/lib/postgresql/data

healthcheck:

test: ['CMD-SHELL', 'pg_isready -U n8n']

interval: 10s

timeout: 5s

retries: 5

n8n:

image: n8nio/n8n:latest

restart: always

ports:

- '5678:5678'

environment:

- N8N_BASIC_AUTH_ACTIVE=true

- N8N_BASIC_AUTH_USER=\${N8N_BASIC_AUTH_USER}

- N8N_BASIC_AUTH_PASSWORD=\${N8N_BASIC_AUTH_PASSWORD}

- N8N_HOST=\${N8N_HOST}

- N8N_PORT=5678

- N8N_PROTOCOL=https

- WEBHOOK_URL=\${WEBHOOK_URL}

- DB_TYPE=postgresdb

- DB_POSTGRESDB_HOST=postgres

- DB_POSTGRESDB_PORT=5432

- DB_POSTGRESDB_DATABASE=n8n

- DB_POSTGRESDB_USER=n8n

- DB_POSTGRESDB_PASSWORD=\${POSTGRES_PASSWORD}

- EXECUTIONS_DATA_SAVE_ON_SUCCESS=all

- EXECUTIONS_DATA_SAVE_ON_ERROR=all

volumes:

- n8n-data:/home/node/.n8n

- ./workflows:/workflows

depends_on:

postgres:

condition: service_healthy

links:

- postgres

volumes:

postgres-data:

```
n8n-data:
EOF
```

Step 4: Create .env file

```
cat > .env << 'EOF'
POSTGRES_PASSWORD=your_secure_postgres_password
N8N_BASIC_AUTH_USER=admin
N8N_BASIC_AUTH_PASSWORD=your_secure_n8n_password
N8N_HOST=your-domain.com
WEBHOOK_URL=https://your-domain.com
EOF
```

Step 5: Create workflows directory

```
mkdir workflows
```

Step 6: Start services

```
docker-compose up -d
```

Check status

```
docker-compose ps
docker-compose logs -f n8n
```

Step 7: Setup Nginx reverse proxy (same as self-hosted option)

Follow Step 9-11 from Option 2

Step 8: Access N8N and import workflows

https://your-domain.com

Deployment time: 2-4 hours

7.3 API Credentials Configuration

yaml

OpenAI API Configuration:

1. Obtain API Key:

- Visit: <https://platform.openai.com/api-keys>
- Click "Create new secret key"
- Copy key (starts with sk-...)
- Store securely (cannot retrieve later)

2. Add to N8N:

- Navigate to: Settings → Credentials → New
- Select: OpenAI
- Name: "OpenAI API"

- **API Key:** [paste your key]
- Save

3. Configure in workflows:

- All agent nodes use this credential
- **Model:** gpt-4o
- **Temperature:** 0.6-0.75 (varies by agent)
- **Max tokens:** 1500-4000

4. Monitor usage:

- **Dashboard:** <https://platform.openai.com/usage>
- Set usage limits to prevent overruns
- **Recommended:** \$100/month limit for testing
- **Production:** \$500-1000/month for 500 analyses

Anthropic API Configuration:

1. Obtain API Key:

- Visit: <https://console.anthropic.com/>
- **Navigate to:** Settings → API Keys
- Click "Create Key"
- Copy key (starts with sk-ant-...)

2. Add to N8N:

- Settings → Credentials → New
- **Select:** Anthropic
- **Name:** "Anthropic API"
- **API Key:** [paste your key]
- Save

3. Configure in Chat Assistant workflow:

- **Model:** claude-3-5-sonnet-20241022
- **Temperature:** 0.7
- **Max tokens:** 2000

Google Cloud Setup:

1. Create Project:

- Visit: <https://console.cloud.google.com/>
- Click "Select a project" → "New Project"
- **Project name:** "StratAI"
- Click "Create"

2. Enable APIs:

- **Navigate to:** APIs & Services → Library
- **Enable the following:**

- * Google Docs API
- * Google Drive API
- * Gmail API

3. Create OAuth 2.0 Credentials:

- APIs & Services → Credentials
- Click "Create Credentials" → "OAuth client ID"
- Application type: "Web application"
- Name: "StratAI N8N"
- Authorized redirect URIs:
 - * <https://your-domain.com/rest/oauth2-credential/callback>
- Click "Create"
- Copy Client ID and Client Secret

4. Add to N8N:

- Settings → Credentials → New
- Select: "Google API OAuth2"
- Name: "Google API"
- Client ID: [paste]
- Client Secret: [paste]
- Scopes:
 - * <https://www.googleapis.com/auth/documents>
 - * <https://www.googleapis.com/auth/drive>
 - * <https://www.googleapis.com/auth/gmail.send>
- Click "Connect my account"
- Authorize access
- Save

5. Configure in workflows:

- Google Docs: Use "Google API" credential
- Google Drive: Use "Google API" credential
- Gmail: Use "Google API" credential

Airtable Configuration:

1. Create Account:

- Visit: <https://airtable.com/signup>
- Sign up for free account

2. Create Base:

- Click "Add a base" → "Start from scratch"
- Name: "StratAI Production"
- Create tables as per schema (section 4.2)

3. Obtain Personal Access Token:

- Click profile icon → "Developer hub"
- Click "Create token"
- **Name:** "StratAI N8N Integration"
- **Scopes:**
 - * data.records:read
 - * data.records:write
 - * schema.bases:read
- **Access:** Select "StratAI Production" base
- Click "Create token"
- Copy token (starts with pat...)

4. Get Base ID:

- Open your base
- Click "Help" → "API documentation"
- Base ID shown at top (starts with app...)

5. Add to N8N:

- Settings → Credentials → New
- **Select:** "Airtable Token API"
- **Name:** "Airtable API"
- **Access Token:** [paste token]
- Save

6. Configure in workflows:

- All Airtable nodes
- **Base:** Select "StratAI Production"
- **Table:** Select appropriate table
- **Operation:** append/update/search as needed

Gmail Configuration:

- Uses same Google OAuth2 credential as Google Docs
- No additional setup required
- **Send limits:** 500 emails/day (free), 2000/day (Workspace)

Slack Configuration:

1. Create Slack App:

- Visit: <https://api.slack.com/apps>
- Click "Create New App" → "From scratch"
- **App name:** "StratAI"
- **Workspace:** Select your workspace

2. Configure OAuth Scopes:

- **Navigate to:** OAuth & Permissions
- **Add scopes:**

- * chat:write
- * chat:write.public
- * files:write

3. Install to Workspace:

- Click "Install to Workspace"
- Authorize
- Copy "Bot User OAuth Token" (starts with xoxb-)

4. Add to N8N:

- Settings → Credentials → New
- Select: "Slack OAuth2 API"
- Name: "Slack OAuth2 API"
- Access Token: [paste token]
- Save

Microsoft Teams Configuration:

1. Get Incoming Webhook URL:

- Open Teams
- Navigate to channel
- Click "..." → "Connectors"
- Search "Incoming Webhook"
- Configure
- Name: "StratAI Notifications"
- Upload image (optional)
- Click "Create"
- Copy webhook URL

2. Add to N8N:

- No credential needed
- Paste URL directly in workflow
- Or store in environment variable: TEAMS_WEBHOOK_URL

News API Configuration:

1. Sign up:

- Visit: <https://newsapi.org/register>
- Free tier: 100 requests/day

2. Get API Key:

- After signup, key shown immediately
- Also: <https://newsapi.org/account>

3. Add to N8N:

- Settings → Credentials → New

- Select: "Generic Credential Type"
- Name: "newsApi"
- Fields:
 - * name: "apiKey"
 - * value: [your API key]
- Save

7.4 Environment Variables

bash

Create .env file for N8N

```
cat > .env << 'EOF'
```

N8N Configuration

N8N_BASIC_AUTH_ACTIVE=true

N8N_BASIC_AUTH_USER=admin

N8N_BASIC_AUTH_PASSWORD=your_secure_password_here

N8N_HOST=your-domain.com

N8N_PORT=5678

N8N_PROTOCOL=https

WEBHOOK_URL=https://your-domain.com

Database Configuration

DB_TYPE=postgresdb

DB_POSTGRESDB_HOST=localhost

DB_POSTGRESDB_PORT=5432

DB_POSTGRESDB_DATABASE=n8n

DB_POSTGRESDB_USER=n8n_user

DB_POSTGRESDB_PASSWORD=your_postgres_password_here

Execution Settings

EXECUTIONS_DATA_SAVE_ON_SUCCESS=all

EXECUTIONS_DATA_SAVE_ON_ERROR=all

EXECUTIONS_DATA_SAVE_MANUAL_EXECUTIONS=true

EXECUTIONS_TIMEOUT=300

EXECUTIONS_TIMEOUT_MAX=600

Timezone

GENERIC_TIMEZONE=America/New_York

Log Level

N8N_LOG_LEVEL=info

N8N_LOG_OUTPUT=console,file

N8N_LOG_FILE_LOCATION=/var/log/n8n/

```
# External Services (Optional - for workflow use)
OPENAI_API_KEY=sk-...
ANTHROPIC_API_KEY=sk-ant-...
AIRTABLE_API_KEY=pat...
NEWS_API_KEY=...
TEAMS_WEBHOOK_URL=https://outlook.office.com/webhook/...

# Security
N8N_JWT_SECRET=your_jwt_secret_minimum_32_chars
N8N_ENCRYPTION_KEY=your_encryption_key_minimum_32_chars

# Performance
N8N_PAYLOAD_SIZE_MAX=16
N8N_METRICS=true

# Email Configuration (if using SMTP)
N8N_EMAIL_MODE=smtp
N8N_SMTP_HOST=smtp.gmail.com
N8N_SMTP_PORT=587
N8N_SMTP_USER=noreply@stratai.com
N8N_SMTP_PASS=your_email_password
N8N_SMTP_SENDER=noreply@stratai.com
```

EOF

Secure the .env file

```
chmod 600 .env
```

7.5 Testing Deployment

bash

Test 1: Health Check

```
curl https://your-domain.com/healthz
```

Expected: HTTP 200 OK

Test 2: Webhook Accessibility

```
curl -X POST https://your-domain.com/webhook/stratai-submit \
-H "Content-Type: application/json" \
-d '{
  "user_email": "test@example.com",
  "company_name": "Test Company",
  "job_function": "Test Role",
```

```
"strategic_question": "This is a test question for deployment verification",
"user_ai_chip_rate": 75,
"user_crypto_rate": 50,
"user_ev_rate": 60,
"user_supply_risk": 70,
"user_regulatory": 55,
"user_consumer_rate": 65
}'
```

Expected: JSON response with success: true and report_id

Test 3: Database Connectivity

SSH into server

```
sudo -u postgres psql -d n8n -c "SELECT COUNT(*) FROM execution_entity;"
```

Expected: Number showing execution count

Test 4: Check N8N Logs

Self-hosted:

```
sudo journalctl -u n8n -f
```

Docker:

```
docker-compose logs -f n8n
```

Expected: No error messages, successful workflow executions

Test 5: API Key Validation

In N8N UI:

1. Open any agent workflow

2. Click on OpenAI node

3. Click "Test" button

Expected: Successful connection message

Test 6: Google Integration

In N8N UI:

1. Open main workflow

2. Click on "Create Google Doc" node

3. Click "Test" button

Expected: Document created in Google Drive

Test 7: Airtable Integration

Run test analysis and check Airtable base

Expected: New record in Reports table

Test 8: Email Delivery
Trigger email notification workflow manually
Expected: Email received within 30 seconds

Test 9: Load Test (Optional)
Install Apache Bench
`sudo apt install apache2-utils`

Run load test (100 requests, 10 concurrent)
`ab -n 100 -c 10 -p test-payload.json \`
`-T application/json \`
`https://your-domain.com/webhook/stratai-submit`

Expected:
- 100% success rate
- Average response time < 45 seconds
- No server errors

7.6 Post-Deployment Checklist

yaml

Security:

- ☐ SSL certificate installed and auto-renewing
- ☐ Firewall configured (only ports 80, 443, 22 open)
- ☐ N8N basic auth enabled
- ☐ Database password changed from default
- ☐ .env file permissions set to 600
- ☐ All API keys rotated from test keys
- ☐ Backup strategy implemented

Monitoring:

- ☐ N8N health check endpoint responding
- ☐ Database backup script scheduled (daily)
- ☐ Log rotation configured
- ☐ Disk space monitoring alert set (>80% threshold)
- ☐ CPU/memory monitoring configured
- ☐ Error alerting enabled (email or Slack)

Workflows:

- ☐ All 14 workflows imported successfully
- ☐ Credentials configured for all workflows
- ☐ Airtable base IDs updated in all workflows
- ☐ Test execution successful for each workflow
- ☐ Webhooks activated and responding

- ☐ Scheduled workflows enabled (weekly, monthly)

Integration Testing:

- ☐ **End-to-end test:** Submit → Report → Email
- ☐ PDF upload and parsing tested
- ☐ Chat assistant responding correctly
- ☐ Feedback submission working
- ☐ Analytics dashboard displaying data
- ☐ Low confidence alert triggering correctly
- ☐ Slack/Teams notifications working

Documentation:

- ☐ Webhook URLs documented and shared with team
- ☐ Admin credentials stored securely (password manager)
- ☐ Runbook created for common issues
- ☐ Contact information for support documented
- ☐ Disaster recovery plan documented

Performance:

- ☐ Load test completed successfully
- ☐ Response times acceptable (<60s for analysis)
- ☐ Database queries optimized
- ☐ No memory leaks detected
- ☐ Auto-scaling configured (if applicable)

Compliance:

- ☐ Privacy policy updated with third-party processors
- ☐ Terms of service reflect N8N/OpenAI/Claude usage
- ☐ Data processing agreements signed
- ☐ GDPR compliance verified
- ☐ Data retention policies implemented

8. Monitoring & Observability

8.1 Logging Architecture

yaml

Log Levels:

ERROR: System failures, API errors, data corruption

WARN: Deprecation warnings, configuration issues, low confidence scores

INFO: Workflow executions, user actions, API calls

DEBUG: Detailed execution traces, variable states

TRACE: Full request/response bodies (use sparingly)

Log Sources:

N8N Application Logs:

Location: /var/log/n8n/ or Docker logs

Retention: 30 days

Format: JSON

Example:

```
{
  "timestamp": "2025-01-18T10:30:45.123Z",
  "level": "INFO",
  "workflow": "StratAI - Main Strategic Intelligence",
  "execution_id": "12345",
  "message": "Workflow execution started",
  "user_email": "director@azure.com"
}
```

Webhook Access Logs:

Location: Nginx access logs (/var/log/nginx/access.log)

Retention: 7 days

Format: Combined (IP, timestamp, endpoint, status, response time)

Database Logs:

Location: PostgreSQL logs (/var/log/postgresql/)

Retention: 14 days

Focus: Slow queries (>1s), errors, connection issues

API Provider Logs:

OpenAI: <https://platform.openai.com/usage>

Anthropic: Console logs

Google: Cloud Console logs

Airtable: API usage dashboard

Log Aggregation (Recommended for Production):

Option 1: ELK Stack (Elasticsearch, Logstash, Kibana)

- Centralized log storage
- Full-text search
- Visualization dashboards
- **Setup time:** 4-6 hours

Option 2: Grafana Loki + Promtail

- Lightweight alternative to ELK
- Integrates with Grafana for metrics
- Lower resource usage

- Setup time: 2-3 hours

Option 3: Cloud-based (Datadog, New Relic, Splunk)

- Managed service
- Advanced analytics
- Higher cost (\$50-500/month)
- Setup time: 1 hour

Structured Logging Format:

```
{
  "timestamp": "2025-01-18T10:30:45.123Z",
  "level": "INFO|WARN|ERROR",
  "service": "stratai",
  "workflow": "workflow_name",
  "execution_id": "uuid",
  "node": "node_name",
  "user_email": "user@example.com",
  "company_name": "Company Name",
  "report_id": "RPT-xxx",
  "confidence_score": 85.42,
  "duration_ms": 35000,
  "status": "success|failure",
  "error": "error_message (if applicable)",
  "metadata": {
    "key": "value"
  }
}
```

8.2 Metrics & KPIs

yaml

System Performance Metrics:

Workflow Execution Metrics:

- Total executions per hour/day/week
- Success rate (%)
- Failure rate (%)
- Average execution time (seconds)
- P50, P95, P99 execution times
- Timeout rate

API Performance:

- Requests per minute
- Response time by endpoint
- Error rate by endpoint

- Rate limit hits
- API cost per request

Resource Utilization:

- CPU usage (%)
- Memory usage (MB)
- Disk usage (GB)
- Network I/O (MB/s)
- Database connections count
- Active workflow executions

Business Metrics:

Usage Metrics:

- Reports generated (total, daily, weekly, monthly)
- Unique users (daily, weekly, monthly)
- Reports per user (average)
- PDF uploads (count, total size)
- Chat messages sent
- Feedback submissions

Quality Metrics:

- Average confidence score
- High confidence reports (>75%) percentage
- Low confidence reports (<60%) percentage
- Alerts triggered
- User satisfaction (avg rating)
- Would recommend rate (%)

Engagement Metrics:

- Weekly active users (WAU)
- Monthly active users (MAU)
- User retention rate
- Churn rate
- Time to first report (onboarding)
- Reports per session

Cost Metrics:

- OpenAI API cost (\$/report, total/month)
- Anthropic API cost (\$/chat, total/month)
- Infrastructure cost (\$/month)
- Cost per report (total)
- Revenue per user (if applicable)
- Customer acquisition cost (CAC)

Alert Thresholds:

Critical (Page immediately):

- System down (health check fails)
- Error rate > 10%
- Database connection pool exhausted
- Disk usage > 90%
- API key expired/invalid

Warning (Notify during business hours):

- Error rate > 5%
- Average execution time > 90 seconds
- Confidence score average < 65%
- Disk usage > 80%
- Memory usage > 85%

Info (Daily digest):

- Daily report generation count
- Cost summary
- New user signups
- Feedback submitted

8.3 Monitoring Dashboard Setup

Grafana Dashboard Configuration

yaml

Install Grafana (if self-hosted)

Installation:

Ubuntu:

- `sudo apt-get install -y software-properties-common`
- `sudo add-apt-repository "deb https://packages.grafana.com/oss/deb stable main"`
- `wget -q -O - https://packages.grafana.com/gpg.key | sudo apt-key add -`
- `sudo apt-get update`
- `sudo apt-get install grafana`
- `sudo systemctl enable grafana-server`
- `sudo systemctl start grafana-server`
- **Access:** `http://localhost:3000`
- **Default credentials:** admin/admin

Data Sources:

PostgreSQL (N8N Database):

Type: PostgreSQL

Host: localhost:5432

Database: n8n

User: n8n_user
SSL Mode: require (production)

Airtable (via API):
Type: JSON API
URL: https://api.airtable.com/v0/your_base_id/Reports
Headers: Authorization: Bearer pat_xxx

Prometheus (for system metrics):
Type: Prometheus
URL: <http://localhost:9090>

Dashboard Panels:

Panel 1: Reports Generated (Time Series)

Query:

```
SELECT
  DATE_TRUNC('hour', timestamp) as time,
  COUNT(*) as reports
FROM execution_entity
WHERE workflow_name = 'StratAI - Main Strategic Intelligence'
  AND finished = true
GROUP BY time
ORDER BY time DESC
```

Panel 2: Average Confidence Score (Gauge)

Query:

```
SELECT AVG(confidence_score)
FROM airtable_reports
WHERE timestamp > NOW() - INTERVAL '7 days'
```

Thresholds:

< 60%: Red
60-75%: Yellow
> 75%: Green

Panel 3: Success vs Failure Rate (Pie Chart)

Query:

```
SELECT
  CASE WHEN status = 'success' THEN 'Success' ELSE 'Failure' END as status,
  COUNT(*) as count
FROM execution_entity
WHERE workflow_name LIKE 'StratAI%'
  AND timestamp > NOW() - INTERVAL '24 hours'
GROUP BY status
```

Panel 4: API Cost Tracking (Stacked Bar)

Query:

```
SELECT
  DATE_TRUNC('day', timestamp) as time,
  SUM(openai_cost) as openai,
  SUM(anthropic_cost) as anthropic
FROM api_costs
GROUP BY time
ORDER BY time DESC
```

Panel 5: System Resources (Multi-line Graph)

Metrics:

- node_cpu_usage_percent
- node_memory_usage_percent
- node_disk_usage_percent

Source: Prometheus Node Exporter

Panel 6: Recent Alerts (Table)

Query:

```
SELECT
  report_id,
  alert_type,
  confidence_score,
  timestamp
FROM alerts
ORDER BY timestamp DESC
LIMIT 10
```

Alert Rules:

Rule 1: High Error Rate

Condition: error_rate > 10% for 5 minutes

Notification: Email + Slack

Message: "StratAI error rate critical: {{value}}%"

Rule 2: Low Confidence Trend

Condition: avg_confidence < 65% for 24 hours

Notification: Email

Message: "Average confidence score dropping: {{value}}%"

Rule 3: System Resource Alert

Condition: cpu_usage > 90% OR memory_usage > 90%

Notification: Slack

Message: "System resources critical: CPU {{cpu}}%, Memory {{memory}}%"

Rule 4: API Cost Spike

Condition: `daily_api_cost > $100`

Notification: Email

Message: "API costs unusually high today: `{{value}}`"

Custom N8N Monitoring Workflow

json

```
{
  "name": "StratAI - System Monitoring",
  "nodes": [
    {
      "parameters": {
        "rule": {
          "interval": [{"field": "cronExpression", "expression": "*/5 * * * *"}]
        }
      },
      "name": "Every 5 Minutes",
      "type": "n8n-nodes-base.scheduleTrigger"
    },
    {
      "parameters": {
        "jsCode": "/* Query execution statistics\nconst now = new Date();\nconst fiveMinutesAgo =\nnew Date(now - 5 * 60 * 1000);\n\n// This would query your N8N database\nconst stats = {\n  timestamp: now.toISOString(),\n  executions_last_5min: 12, // Query from DB\n  failures_last_5min: 1,\n  avg_duration_seconds: 42,\n  active_executions: 3\n};\n\nconst\nerrorRate = (stats.failures_last_5min / stats.executions_last_5min) * 100;\n\nreturn {\n  json: {\n    ...stats,\n    error_rate: errorRate,\n    status: errorRate > 10 ? 'critical' : errorRate > 5 ? 'warning'\n    : 'healthy'\n  }\n};"
      },
      "name": "Calculate Metrics",
      "type": "n8n-nodes-base.code"
    },
    {
      "parameters": {
        "conditions": {
          "string": [{"value1": "={{ $json.status }}", "value2": "critical"}]
        }
      },
      "name": "IF Critical",
      "type": "n8n-nodes-base.if"
    },
    {
      "parameters": {
```

```

    "fromEmail": "alerts@stratai.com",
    "toEmail": "admin@stratai.com",
    "subject": "🚨 StratAI System Alert - Critical Error Rate",
    "text": "=Error rate: {{ $json.error_rate }}%\nFailures: {{ $json.failures_last_5min }}\nExecutions: {{ $json.executions_last_5min }}"
  },
  "name": "Send Alert Email",
  "type": "n8n-nodes-base.gmail"
},
{
  "parameters": {
    "operation": "append",
    "base": "your-airtable-base-id",
    "table": "SystemMetrics",
    "columns": {
      "mappingMode": "defineBelow",
      "value": {
        "timestamp": "={{ $json.timestamp }}",
        "executions": "={{ $json.executions_last_5min }}",
        "failures": "={{ $json.failures_last_5min }}",
        "error_rate": "={{ $json.error_rate }}",
        "avg_duration": "={{ $json.avg_duration_seconds }}",
        "status": "={{ $json.status }}"
      }
    }
  },
  "name": "Log to Airtable",
  "type": "n8n-nodes-base.airtable"
}
]
}

```

8.4 Error Tracking & Debugging

yaml

Error Categories:

User Input Errors:

- Invalid email format
- Missing required fields
- PDF too large (>10MB)
- Invalid market signal values

Handling: Return 400 Bad Request with validation details

API Errors:

OpenAI:

- Rate limit exceeded (429)
- Invalid API key (401)
- Model overloaded (503)
- Timeout (after 60s)

Anthropic:

- Rate limit exceeded
- Context length exceeded
- API down

Google:

- Quota exceeded
- Invalid OAuth token
- Document creation failed

Airtable:

- Record limit exceeded
- Invalid field type
- API rate limit

Handling: Retry with exponential backoff (3 attempts)

System Errors:

- Database connection lost
- Disk full
- Memory exhausted
- Network timeout

Handling: Alert immediately, attempt graceful degradation

Workflow Logic Errors:

- Agent returned invalid format
- HHH score parsing failed
- Confidence calculation returned NaN
- Section extraction failed

Handling: Log error, return partial result with warning

Error Logging Format:

```
{  
  "timestamp": "2025-01-18T10:30:45.123Z",  
  "level": "ERROR",  
  "error_type": "APIError",  
  "error_code": "OPENAI_RATE_LIMIT",  
  "workflow": "StratAI - Main Strategic Intelligence",  
  "execution_id": "abc123",  
  "node": "Market Data Agent",  
  "user_email": "director@azure.com",  
  "report_id": "RPT-1737158400123",  
}
```

```

"error_message": "Rate limit exceeded for gpt-4o",
"stack_trace": "...",
"retry_attempt": 1,
"resolution": "Retrying in 60 seconds",
"metadata": {
  "api": "openai",
  "model": "gpt-4o",
  "tokens_requested": 1500
}
}

```

Debugging Tools:

N8N Built-in Debugger:

- View execution logs
- Inspect node inputs/outputs
- Replay failed executions
- Download execution data as JSON

Log Analysis Commands:

Search for errors in last hour

```
grep -i error /var/log/n8n/n8n.log | grep "$(date -d '1 hour ago' '+%Y-%m-%d %H')"
```

Count errors by type

```
grep ERROR /var/log/n8n/n8n.log | jq -r '.error_type' | sort | uniq -c
```

Find slow executions (>60s)

```
grep duration_ms /var/log/n8n/n8n.log | jq 'select(.duration_ms > 60000)'
```

Monitor live errors

```
tail -f /var/log/n8n/n8n.log | grep ERROR
```

Database Debugging:

Check failed executions

```

SELECT
  id,
  workflow_name,
  error,
  started_at,
  stopped_at,
  EXTRACT(EPOCH FROM (stopped_at - started_at)) as duration_seconds
FROM execution_entity
WHERE finished = false OR error IS NOT NULL
ORDER BY started_at DESC
LIMIT 20;

```

Find workflows with high failure rate

```
SELECT
  workflow_name,
  COUNT(*) as total,
  SUM(CASE WHEN error IS NOT NULL THEN 1 ELSE 0 END) as failures,
  ROUND(100.0 * SUM(CASE WHEN error IS NOT NULL THEN 1 ELSE 0 END) / COUNT(*),
```

2) as failure_rate

```
FROM execution_entity
WHERE started_at > NOW() - INTERVAL '24 hours'
GROUP BY workflow_name
ORDER BY failure_rate DESC;
```

Common Issues & Solutions:

Issue: "OpenAI rate limit exceeded"

Solution:

- Implement exponential backoff
- Reduce concurrent executions
- Upgrade OpenAI plan
- Cache responses where possible

Issue: "Confidence score returning NaN"

Solution:

- Check HHH score format in synthesizer output
- Verify regex pattern matches agent output
- Add error handling in JavaScript calculator
- Log synthesizer output for inspection

Issue: "Google Doc creation timeout"

Solution:

- Increase N8N timeout setting
- Check Google API quotas
- Verify OAuth token is valid
- Reduce document size if >50 pages

Issue: "Airtable record limit exceeded"

Solution:

- Archive old records
- Upgrade Airtable plan
- Implement record cleanup workflow
- Use linked tables for relationships

Issue: "PDF parsing failed"

Solution:

- Check PDF file size (<10MB)
- Verify PDF is not encrypted
- Try re-uploading PDF
- Check pdf-parse library version

9. Disaster Recovery

9.1 Backup Strategy

yaml

Database Backups:

PostgreSQL (N8N Data):

Frequency: Daily at 2 AM UTC

Retention: 30 days

Location:

- Local: /var/backups/n8n/
- Remote: AWS S3 / Google Cloud Storage

Backup Script:

```
#!/bin/bash
```

```
# /usr/local/bin/backup-n8n.sh
```

```
BACKUP_DIR="/var/backups/n8n"
```

```
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
```

```
BACKUP_FILE="n8n_backup_${TIMESTAMP}.sql.gz"
```

```
# Create backup directory if it doesn't exist
```

```
mkdir -p $BACKUP_DIR
```

```
# Create backup
```

```
pg_dump -U n8n_user n8n | gzip > "${BACKUP_DIR}/${BACKUP_FILE}"
```

```
# Upload to S3 (if configured)
```

```
if [ -n "$AWS_S3_BUCKET" ]; then
```

```
aws s3 cp "${BACKUP_DIR}/${BACKUP_FILE}" \
```

```
"s3://${AWS_S3_BUCKET}/n8n-backups/"
```

```
fi
```

```
# Delete backups older than 30 days
```

```
find $BACKUP_DIR -name "n8n_backup_*.sql.gz" -mtime +30 -delete
```

```
echo "Backup completed: ${BACKUP_FILE}"
```

Cron Schedule:

```
0 2 * * * /usr/local/bin/backup-n8n.sh >> /var/log/n8n-backup.log 2>&1
```

Restore Procedure:

Stop N8N

```
sudo systemctl stop n8n
```

Restore database

```
gunzip < n8n_backup_20250118_020000.sql.gz | psql -U n8n_user -d n8n
```

Start N8N

```
sudo systemctl start n8n
```

Verify workflows are intact

```
curl https://your-domain.com/healthz
```

Airtable Backups:

Frequency: Daily via API

Method: Export to CSV via N8N workflow

Backup Workflow:

nodes:

- Schedule Trigger (daily at 3 AM)
- **Airtable**: List all records from Reports table
- **Airtable**: List all records from Feedback table
- **Airtable**: List all records from Users table
- **Code**: Convert to CSV format
- **Google Drive**: Upload CSV files
- **Email**: Send backup completion notification

Workflow Backups:

Frequency: On every change (manual)

Method: Export workflow JSON

Location: Git repository

Export Process:

1. Settings → Workflows
2. Select workflow → Export
3. Save JSON file
4. **Commit to Git**: git add .; git commit -m "Backup workflows"; git push

Automated Export Script:

```
#!/bin/bash
```

```
# Export all N8N workflows via API
```

```

N8N_URL="https://your-domain.com"
N8N_USER="admin"
N8N_PASS="password"
BACKUP_DIR="./workflow-backups"

mkdir -p $BACKUP_DIR

# Get all workflows
WORKFLOWS=$(curl -u $N8N_USER:$N8N_PASS \
  "${N8N_URL}/rest/workflows" | jq -r '.[].id')

# Export each workflow
for ID in $WORKFLOWS; do
  curl -u $N8N_USER:$N8N_PASS \
    "${N8N_URL}/rest/workflows/${ID}" \
    > "${BACKUP_DIR}/workflow_${ID}.json"
done

echo "Exported $(echo $WORKFLOWS | wc -w) workflows"

```

Configuration Backups:

Files to Backup:

- .env file (encrypted)
- nginx configuration (/etc/nginx/sites-available/)
- systemd service file (/etc/systemd/system/n8n.service)
- SSL certificates (/etc/letsencrypt/)
- Custom scripts (/usr/local/bin/)

Backup Method:

Create encrypted tar archive

```
tar czf - .env /etc/nginx/sites-available/ /etc/systemd/system/n8n.service | \
  openssl enc -aes-256-cbc -salt -out config_backup.tar.gz.enc
```

Upload to secure storage

```
aws s3 cp config_backup.tar.gz.enc s3://your-bucket/config-backups/
```

Restore:

Download and decrypt

```
aws s3 cp s3://your-bucket/config-backups/config_backup.tar.gz.enc .
openssl enc -aes-256-cbc -d -in config_backup.tar.gz.enc | tar xz
```

9.2 Recovery Procedures

yaml

Scenario 1: Database Corruption

Symptoms:

- N8N fails to start
- Error: "Database connection failed"
- Workflows not loading

Recovery Steps:

1. Stop N8N service
`sudo systemctl stop n8n`
2. Check database status
`sudo systemctl status postgresql`
`sudo -u postgres psql -c "SELECT version();"`
3. Attempt database repair
`sudo -u postgres vacuumdb --all --full`
4. If repair fails, restore from backup
`sudo -u postgres dropdb n8n`
`sudo -u postgres createdb n8n`
`gunzip < latest_backup.sql.gz | sudo -u postgres psql n8n`
5. Verify data integrity
`sudo -u postgres psql n8n -c "SELECT COUNT(*) FROM workflow_entity;"`
6. Restart N8N
`sudo systemctl start n8n`
`sudo systemctl status n8n`
7. Test workflows
`curl -X POST https://your-domain.com/webhook-test`

Expected Recovery Time: 15-30 minutes

Data Loss: None (if backup recent)

Scenario 2: Workflow Execution Failure

Symptoms:

- Workflows timing out
- All executions failing
- High error rate in logs

Recovery Steps:

1. Identify failing workflows

```
SELECT workflow_name, COUNT(*) as failures
FROM execution_entity
WHERE error IS NOT NULL AND started_at > NOW() - INTERVAL '1 hour'
GROUP BY workflow_name;
```

2. Check system resources

```
htop
df -h
free -m
```

3. Review recent changes

```
git log --since="1 day ago"
```

4. Rollback to last known good version

- Export current workflows (backup)
- Import previous workflow versions from Git
- Test execution

5. If API issues, verify credentials

- Check OpenAI API key validity
- Verify Google OAuth token
- Test Airtable connection

6. Clear execution queue if stuck

```
DELETE FROM execution_entity WHERE finished = false;
```

7. Restart N8N with clean slate

```
sudo systemctl restart n8n
```

Expected Recovery Time: 5-15 minutes

Data Loss: In-progress executions lost

Scenario 3: Complete Server Failure

Symptoms:

- Server unreachable
- SSH connection refused
- Website down

Recovery Steps:

1. Provision new server
 - Same specs as original
 - Same OS version (Ubuntu 22.04)
2. Install base software

- Node.js 18.x
- PostgreSQL 14
- Nginx
- N8N

3. Restore database from backup

```
scp backup-server:/backups/n8n_latest.sql.gz .  
gunzip < n8n_latest.sql.gz | psql -U n8n_user -d n8n
```

4. Restore configuration files

```
scp backup-server:/backups/config_backup.tar.gz.enc .  
openssl enc -aes-256-cbc -d -in config_backup.tar.gz.enc | tar xz  
sudo cp -r etc/* /etc/
```

5. Update DNS to point to new server
(if IP changed)

6. Obtain new SSL certificate

```
sudo certbot --nginx -d your-domain.com
```

7. Start services

```
sudo systemctl start postgresql  
sudo systemctl start n8n  
sudo systemctl start nginx
```

8. Verify all workflows functional

Run end-to-end test

Expected Recovery Time: 2-4 hours

Data Loss: Up to last backup (24 hours max)

Scenario 4: API Key Compromise

Symptoms:

- Unauthorized API usage
- Unusual cost spike
- Security alert from provider

Recovery Steps:

1. Immediately revoke compromised keys

- **OpenAI:** platform.openai.com/api-keys
- **Anthropic:** console.anthropic.com
- **Google:** console.cloud.google.com
- **Airtable:** airtable.com/account

2. Generate new API keys
 - Create new keys with restricted permissions
 - Document rotation in password manager
3. Update N8N credentials
 - Settings → Credentials
 - Update each affected credential
 - Test connections
4. Review audit logs
 - Check OpenAI usage logs for unauthorized requests
 - Review N8N execution logs for suspicious activity
 - Identify extent of compromise
5. Notify affected users if data exposed
 - Email notification
 - Incident report
6. Implement additional security
 - Enable IP whitelisting on APIs
 - Add rate limiting
 - Review IAM permissions
7. Document incident
 - Root cause analysis
 - Timeline of events
 - Preventive measures

Expected Recovery Time: 1-2 hours

Data Loss: None (keys rotated, no data lost)

Scenario 5: Airtable Data Loss

Symptoms:

- Missing records in Airtable
- Accidental deletion
- Data corruption

Recovery Steps:

1. Check Airtable revision history
 - Airtable keeps 14-day revision history (free tier)
 - Restore from revision if available
2. If beyond revision window, restore from CSV backup
 - Download latest backup from Google Drive

- Import CSV into Airtable
- Verify record counts match

3. Reconcile with N8N database

```
SELECT COUNT(*) FROM execution_entity  
WHERE workflow_name = 'StratAI - Main Strategic Intelligence';
```

Compare with Airtable record count

4. Re-run workflows if needed

- Identify missing reports by date range
- Manually trigger workflows for missing periods

5. Implement preventive measures

- Increase backup frequency
- Add record-level audit logging
- Restrict deletion permissions

Expected Recovery Time: 30-60 minutes

Data Loss: Minimal (up to last backup)

9.3 Business Continuity Plan

yaml

Critical System Components:

Tier 1 (Mission Critical):

- N8N orchestration engine
- PostgreSQL database
- OpenAI API access
- Google Docs API
- Airtable storage

RTO: 1 hour

RPO: 1 hour

Tier 2 (Important):

- Anthropic Claude API (Chat)
- Email notifications
- Slack integrations
- Scheduled reports

RTO: 4 hours

RPO: 24 hours

Tier 3 (Nice to Have):

- Analytics dashboard

- Competitor monitoring
- PDF export
- Teams notifications

RTO: 24 hours

RPO: 7 days

Failover Strategy:

Primary Region: US-East (Virginia)

Secondary Region: US-West (Oregon)

Failover Triggers:

- Primary region outage > 15 minutes
- Error rate > 50% for 10 minutes
- Database unreachable for 5 minutes
- Manual failover initiated by admin

Automated Failover (if implemented):

- DNS failover with Route53 health checks
- Database replication to secondary
- Workflow sync between regions
- Automated testing of secondary every 24 hours

Manual Failover Process:

1. Verify primary is actually down (not transient)
2. Update DNS to point to secondary server
3. Promote secondary database to primary
4. Verify all workflows operational
5. Notify users of any expected disruptions
6. Monitor secondary closely
7. Prepare primary for return to service

Expected Failover Time: 10-20 minutes (manual)

Communication Plan:

Internal Communications:

- Slack *#incidents channel*
- Email to engineering@stratai.com
- PagerDuty alerts for on-call engineer

External Communications:

- Status page: status.stratai.com
- Twitter: [@StratAI_Status](https://twitter.com/StratAI_Status)
- Email to active users (if outage >30 min)
- In-app banner notification

Communication Templates:

Incident Start:

"We're currently experiencing issues with StratAI strategic analysis generation. Our team is investigating and will provide updates every 30 minutes. Current status: [brief description]"

Incident Update:

"Update: We've identified the root cause as [issue]. Our team is working on a fix. Estimated resolution: [time]. Affected features: [list]"

Incident Resolved:

"StratAI is fully operational. All systems have been restored. Root cause: [brief explanation]. We apologize for the disruption. Post-mortem will be published within 48 hours."

Testing Schedule:

Disaster Recovery Drills:

Frequency: Quarterly

Scope: Full system recovery from backup

Duration: 2-3 hours

Participants: Engineering team + 1 executive sponsor

Success Criteria:

- Database restored within 30 minutes
- All workflows operational within 60 minutes
- End-to-end test successful
- Documentation updated with lessons learned

Backup Verification:

Frequency: Monthly

Process:

1. Restore latest backup to test environment
2. Verify workflow functionality
3. Check data integrity (record counts match)
4. Test sample analysis end-to-end
5. Document any issues found

Failover Testing:

Frequency: Semi-annually

Process:

1. Schedule maintenance window
2. Execute manual failover to secondary
3. Verify all systems operational

4. Run load test
5. Failback to primary
6. Document timing and issues

10. Performance Optimization

10.1 Database Optimization

sql

-- PostgreSQL Performance Tuning

-- 1. Create indexes for common queries

```
CREATE INDEX idx_execution_workflow ON execution_entity(workflow_name, started_at  
DESC);
```

```
CREATE INDEX idx_execution_status ON execution_entity(finished, error);
```

```
CREATE INDEX idx_execution_user ON execution_entity((data->'user_email'), started_at  
DESC);
```

-- 2. Analyze and vacuum regularly

```
VACUUM ANALYZE execution_entity;
```

```
VACUUM ANALYZE workflow_entity;
```

-- 3. Clean up old executions (run monthly)

```
DELETE FROM execution_entity
```

```
WHERE finished = true
```

```
AND stopped_at < NOW() - INTERVAL '90 days';
```

-- 4. Optimize PostgreSQL configuration (/etc/postgresql/14/main/postgresql.conf)

```
shared_buffers = 256MB      # 25% of RAM
```

```
effective_cache_size = 1GB   # 50-75% of RAM
```

```
work_mem = 16MB              # RAM / max_connections / 2
```

```
maintenance_work_mem = 128MB # RAM / 16
```

```
max_connections = 100
```

```
random_page_cost = 1.1      # For SSD storage
```

-- 5. Monitor slow queries

-- Enable slow query logging

```
log_min_duration_statement = 1000 # Log queries > 1 second
```

-- Query to find slow queries

```
SELECT
```

```
query,
```

```
calls,
```

```
total_time,  
mean_time,  
max_time  
FROM pg_stat_statements  
ORDER BY mean_time DESC  
LIMIT 20;
```

-- 6. Connection pooling with PgBouncer (optional)

Install PgBouncer

```
sudo apt install pgbouncer
```

Configure /etc/pgbouncer/pgbouncer.ini

[databases]

```
n8n = host=localhost port=5432 dbname=n8n
```

[pgbouncer]

```
listen_addr = 127.0.0.1
```

```
listen_port = 6432
```

```
auth_type = md5
```

```
auth_file = /etc/pgbouncer/userlist.txt
```

```
pool_mode = transaction
```

```
max_client_conn = 200
```

```
default_pool_size = 25
```

Update N8N to use PgBouncer

```
DB_POSTGRESDB_PORT=6432
```

10.2 N8N Performance Tuning

yaml

N8N Configuration Optimizations:

Execution Settings:

Increase concurrent executions

```
EXECUTIONS_PROCESS=main
```

```
EXECUTIONS_MODE=regular
```

Adjust timeouts

```
EXECUTIONS_TIMEOUT=300 # 5 minutes default
```

```
EXECUTIONS_TIMEOUT_MAX=600 # 10 minutes maximum
```

Data pruning

```
EXECUTIONS_DATA_PRUNE=true
```

```
EXECUTIONS_DATA_MAX_AGE=336 # 14 days (in hours)
```

Performance settings

```
N8N_PAYLOAD_SIZE_MAX=16      # Max MB for payloads
NODE_FUNCTION_ALLOW_BUILTIN=*  # Allow all built-in modules
NODE_FUNCTION_ALLOW_EXTERNAL=  # Restrict external modules
```

Memory Management:

Node.js memory limits

```
NODE_OPTIONS=--max-old-space-size=4096 # 4GB heap
```

Garbage collection tuning

```
NODE_OPTIONS=--max-old-space-size=4096 --expose-gc
```

Caching:

Enable workflow caching

```
WORKFLOWS_CACHE_ENABLED=true
```

Cache static assets

```
N8N_STATIC_CACHE_ENABLED=true
```

Workflow Optimizations:

1. Parallel Execution:

Instead of sequential API calls

Agent 1 → Agent 2 → Agent 3 → Agent 4 → Agent 5

Use parallel execution

```
┌→ Agent 1 ┐
├→ Agent 2 ┤→ Merge → Synthesizer
├→ Agent 3 ┤
├→ Agent 4 ┤
└→ Agent 5 ┘
```

Time savings: 5x (25s vs 125s)

2. Reduce API Calls:

Cache frequently used data

- Store market context in variables
- Reuse agent responses when possible
- Batch Airtable operations

3. Optimize Code Nodes:

Bad: Inefficient loops

```
for (let i = 0; i < items.length; i++) {
```

```

    for (let j = 0; j < items.length; j++) {
      // O(n²) complexity
    }
  }
}

```

Good: Use efficient methods

```
items.map(item => processItem(item)); // O(n)
```

4. Limit Data Transfer:

Only pass necessary data between nodes

```

return {
  json: {
    report_id: $json.report_id,      // ✓ Need this
    confidence_score: $json.confidence, // ✓ Need this
    // full_report: $json.full_report // ✗ Don't need in next node
  }
};

```

5. Use Pagination:

When querying large datasets from Airtable

```

options: {
  pageSize: 100, // Max 100 records per page
  maxRecords: 1000 // Limit total records
}

```

Resource Monitoring Script:

```
#!/bin/bash
```

```
# /usr/local/bin/monitor-n8n.sh
```

CPU usage

```
CPU=$(top -bn1 | grep "Cpu(s)" | awk '{print $2}' | cut -d'%' -f1)
```

Memory usage

```
MEM=$(free | grep Mem | awk '{print ($3/$2) * 100.0}')
```

Disk usage

```
DISK=$(df -h / | awk 'NR==2 {print $5}' | cut -d'%' -f1)
```

Active N8N executions

```

EXECUTIONS=$(psql -U n8n_user -d n8n -t -c \
  "SELECT COUNT(*) FROM execution_entity WHERE finished = false;")

```

Log metrics

```
echo "$(date) | CPU: ${CPU}% | Memory: ${MEM}% | Disk: ${DISK}% | Active:
${EXECUTIONS}" \
>> /var/log/n8n-metrics.log
```

Alert if thresholds exceeded

```
if (( $(echo "$CPU > 80" | bc -l) )) || \
    (( $(echo "$MEM > 85" | bc -l) )) || \
    (( DISK > 85 )); then
    echo "Resource alert: CPU=${CPU}%, MEM=${MEM}%, DISK=${DISK}%" | \
    mail -s "N8N Resource Alert" admin@stratai.com
fi
```

Run every 5 minutes

*# Cron: */5 * * * * /usr/local/bin/monitor-n8n.sh*

10.3 API Optimization (Continued)

yaml

OpenAI API Optimization (continued):

Rate Limits:

GPT-4o (Tier 1):

- 500 requests per minute (RPM)
- 30,000 tokens per minute (TPM)
- 10,000 requests per day (RPD)

GPT-4o (Tier 5 - Production):

- 10,000 RPM
- 2,000,000 TPM
- Unlimited daily requests

Optimization Strategies:

1. Token Usage Reduction:

Minimize system prompts

Before: 1,200 tokens

After: 800 tokens (33% reduction)

Savings: \$0.012 per request

Use concise user messages

- Avoid repeating context
- Use abbreviations where clear
- Remove unnecessary examples

2. Request Batching:

Instead of 5 sequential calls

for agent in agents:

 response = openai.call(agent)

Use parallel execution in N8N

All 5 agents triggered simultaneously

Completion time: 25s vs 125s (5x faster)

3. Caching Strategy:

Cache market context (updated weekly)

market_context = get_from_cache('market_context')

if not market_context:

 market_context = generate_market_context()

 set_cache('market_context', market_context, ttl=604800)

Savings: ~500 tokens per request

4. Retry Logic with Exponential Backoff:

max_retries = 3

base_delay = 1 *# second*

for attempt in range(max_retries):

 try:

 response = openai.call()

 break

 except RateLimitError:

 if attempt < max_retries - 1:

 wait_time = base_delay * (2 ** attempt)

 sleep(wait_time)

 else:

 raise

5. Model Selection:

Use Case → Model → Cost

- Agents (complex analysis) → gpt-4o → \$0.025/1K tokens

- Chat (simple Q&A) → gpt-3.5-turbo → \$0.002/1K tokens

- Summary (basic) → gpt-3.5-turbo → \$0.002/1K tokens

Consider: Use gpt-3.5-turbo for chat to reduce costs by 92%

Anthropic API Optimization:

Rate Limits:

Claude 3.5 Sonnet (Tier 1):

- 50 requests per minute
- 40,000 tokens per minute
- 1,000 requests per day

Claude 3.5 Sonnet (Tier 4):

- 4,000 RPM
- 400,000 TPM
- Unlimited daily

Optimization Strategies:

1. Prompt Caching (Beta Feature):

Cache expensive context

```
system_prompt = """
```

```
You are StratAI Chat Assistant...
```

```
[Long context - 5000 tokens]
```

```
"""
```

```
response = anthropic.messages.create(  
    model="claude-3-5-sonnet-20241022",  
    system=[  
        {  
            "type": "text",  
            "text": system_prompt,  
            "cache_control": {"type": "ephemeral"} # Cache this  
        }  
    ]  
)
```

Savings: 90% reduction on cached tokens

2. Reduce Max Tokens:

Chat responses rarely need 2000 tokens

max_tokens: 1000 (instead of 2000)

Cost reduction: 50%

Google APIs Optimization:

Quotas:

Google Docs API:

- 300 requests per minute per user
- 60 requests per minute per project

Google Drive API:

- 12,000 requests per minute per user
- 1,000 requests per minute per project

Optimization:

1. Batch Operations:

Instead of multiple single requests

```
for doc_id in doc_ids:
    get_document(doc_id)
```

Use batch API

```
batch = service.new_batch_http_request()
for doc_id in doc_ids:
    batch.add(service.documents().get(documentId=doc_id))
batch.execute()
```

Performance: 10x faster

2. Use Partial Responses:

Only request fields you need

```
service.documents().get(
    documentId=doc_id,
    fields="title,body/content" # Not entire document
)
```

3. Implement Exponential Backoff:

```
from googleapiclient.errors import HttpError

for n in range(5):
    try:
        result = service.documents().create(body=doc).execute()
        break
    except HttpError as e:
        if e.resp.status in [403, 429, 500, 503]:
            time.sleep((2 ** n) + random.random())
        else:
            raise
```

Airtable API Optimization:

Rate Limits:

Free Plan:

- 5 requests per second per base
- 50,000 API calls per workspace per month

Plus Plan:

- 5 requests per second per base
- 100,000 API calls per workspace per month

Pro Plan:

- 10 requests per second per base
- 200,000 API calls per workspace per month

Optimization:

1. Batch Record Operations:

Create up to 10 records per request

```
records = [  
    {"fields": {...}},  
    {"fields": {...}},  
    # ... up to 10 records  
]
```

```
airtable.create(records)
```

Reduction: 10x fewer API calls

2. Use filterByFormula:

Instead of fetching all and filtering in code

```
all_records = airtable.get_all()  
filtered = [r for r in all_records if r['confidence'] > 75]
```

Filter server-side

```
records = airtable.get_all(  
    formula="AND({confidence_score} > 75)"  
)
```

Performance: 5-10x faster for large datasets

3. Selective Field Retrieval:

Only fetch fields you need

```
records = airtable.get_all(  
    fields=['report_id', 'confidence_score', 'timestamp']  
)
```

Bandwidth reduction: 80%

4. Implement Caching:

Cache frequently accessed records

```
from functools import lru_cache
```

```

@lru_cache(maxsize=100)
def get_report(report_id):
    return airtable.get(report_id)

# Invalidate cache after updates
get_report.cache_clear()

```

Connection Pooling:

HTTP Keep-Alive:

N8N automatically uses connection pooling
Verify it's enabled:

```

import http from 'k6/http';

const params = {
  headers: {
    'Connection': 'keep-alive'
  }
};

```

Benefits:

- Reduce connection overhead by 30-50%
- Lower latency for subsequent requests
- Fewer TCP handshakes

Database Connection Pool:

PostgreSQL pooling (already configured)
 max_connections = 100

PgBouncer settings
 default_pool_size = 25
 max_client_conn = 200
 pool_mode = transaction

Benefits:

- Handle 200 clients with 25 DB connections
- Reduce connection overhead
- Better resource utilization

10.4 Caching Strategy

yaml

Multi-Layer Caching:

Layer 1: In-Memory Cache (Application Level)

Technology: Node.js Map or Redis

TTL: 5-60 minutes

Use Cases:

- Market context (60 min)
- User session data (30 min)
- Recent report metadata (15 min)

Implementation:

```
const cache = new Map();

function getCached(key, ttl, generator) {
  const cached = cache.get(key);
  if (cached && Date.now() - cached.timestamp < ttl) {
    return cached.value;
  }

  const value = generator();
  cache.set(key, {
    value: value,
    timestamp: Date.now()
  });

  return value;
}

// Usage
const marketContext = getCached(
  'market_context',
  3600000, // 1 hour
  () => generateMarketContext()
);
```

Layer 2: Database Cache (Materialized Views)

Technology: PostgreSQL Materialized Views

Refresh: Hourly

Use Cases:

- Analytics aggregations
- Report statistics
- User metrics

Implementation:

```

-- Create materialized view for analytics
CREATE MATERIALIZED VIEW daily_report_stats AS
SELECT
  DATE(timestamp) as date,
  COUNT(*) as total_reports,
  AVG(confidence_score) as avg_confidence,
  COUNT(DISTINCT user_email) as unique_users
FROM airtable_reports
GROUP BY DATE(timestamp);

-- Refresh hourly via cron
REFRESH MATERIALIZED VIEW daily_report_stats;

-- Create index for fast queries
CREATE INDEX idx_daily_stats_date
ON daily_report_stats(date DESC);

```

Layer 3: CDN Cache (Static Assets)

Technology: Cloudflare or CloudFront

TTL: 24 hours

Use Cases:

- JavaScript/CSS files
- Images and icons
- Static JSON responses

Configuration (Cloudflare):

Page Rules:

- **Cache Level:** Standard
- **Browser Cache TTL:** 4 hours
- **Edge Cache TTL:** 24 hours

Cache-Control Headers:

Cache-Control: public, max-age=86400, s-maxage=86400

Layer 4: API Response Cache

Technology: N8N built-in caching

TTL: Varies by endpoint

Use Cases:

- Analytics endpoint (5 min)
- User profile (15 min)
- Report list (1 min)

Implementation:

In N8N HTTP Response node

```
options:
  response:
    headers:
      Cache-Control: 'public, max-age=300' # 5 minutes
      ETag: '{{ $json.report_id }}-{{ $json.timestamp }}'
```

Cache Invalidation Strategy:

Event-Based Invalidation:

Clear cache when data changes

```
events = {
  'report_created': ['recent_reports', 'user_stats'],
  'feedback_submitted': ['report_feedback', 'avg_rating'],
  'user_updated': ['user_profile']
}
```

```
function invalidateCache(event) {
  const keys = events[event];
  keys.forEach(key => cache.delete(key));
}
```

Time-Based Invalidation:

Automatic expiration

```
setInterval(() => {
  const now = Date.now();
  for (let [key, value] of cache) {
    if (now - value.timestamp > value.ttl) {
      cache.delete(key);
    }
  }
}, 60000); // Check every minute
```

Manual Invalidation:

Admin endpoint to clear cache

```
POST /api/admin/clear-cache
{
  "keys": ["market_context", "analytics"]
}
```

10.5 Load Testing

yaml

Current Capacity (Single Server):

- 500 reports/day

- 20 concurrent users
- 2,000 chat messages/day
- 50 GB storage

Scale to 5,000 reports/day:

Phase 1: Vertical Scaling (Month 1)

Actions:

- Upgrade server: 8 vCPU, 16GB RAM
- Add Redis cache (4GB)
- Optimize database queries
- Implement connection pooling

Expected Capacity:

- 2,000 reports/day
- 40 concurrent users
- 8,000 chat messages/day

Cost: +\$100/month

Phase 2: Database Optimization (Month 2)

Actions:

- Separate database server (4 vCPU, 8GB RAM)
- Implement read replicas
- Add database caching layer
- Optimize indexes

Expected Capacity:

- 3,500 reports/day
- 60 concurrent users
- 12,000 chat messages/day

Cost: +\$150/month

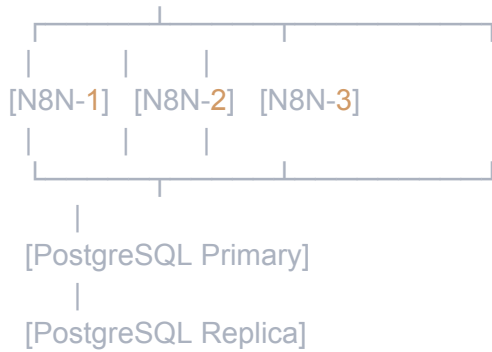
Phase 3: Horizontal Scaling (Month 3-4)

Actions:

- Deploy 2-3 N8N instances behind load balancer
- Shared PostgreSQL (high availability)
- Distributed Redis cache
- Implement sticky sessions

Architecture:

[Load Balancer]
|



Expected Capacity:

- 5,000+ reports/day
- 100+ concurrent users
- 20,000+ chat messages/day

Cost: +\$400/month

Scale to 50,000 reports/day (Enterprise):

Architecture: Microservices

Components:

- API Gateway (Kong/AWS API Gateway)
- N8N Cluster (5-10 instances)
- Dedicated agent services (containerized)
- Message queue (RabbitMQ/AWS SQS)
- Database cluster (PostgreSQL + read replicas)
- Cache cluster (Redis Cluster)
- Object storage (S3/GCS)
- CDN (CloudFront/Cloudflare)

Deployment:

- Kubernetes (EKS/GKE)
- Auto-scaling groups
- Multi-region (US-East, US-West, EU)
- Global load balancing

Expected Capacity:

- 50,000+ reports/day
- 500+ concurrent users
- 200,000+ chat messages/day
- 99.9% uptime SLA

Cost: \$3,000-5,000/month

Migration Path:

1. Containerize N8N workflows
2. Set up Kubernetes cluster
3. Deploy microservices gradually
4. Implement blue-green deployment
5. Migrate users in batches
6. Monitor and optimize

11. Appendix

11.1 Glossary

yaml

Terms:

Agent:

Definition: A specialized AI model (GPT-4o instance) with a specific system prompt focused on one analysis domain

Example: Market Data Agent, Risk Management Agent

Airtable Base:

Definition: A database containing related tables in Airtable

Example: StratAI Production base with Reports, Feedback, Users tables

Confidence Score:

Definition: A numerical value (0-100) representing the reliability of the strategic analysis, calculated from HHH scores

Formula: (Weighted HHH Average) + (Adjustment)

HHH Framework:

Definition: Evaluation methodology assessing AI outputs on three dimensions: Helpful, Harmless, Honest

Purpose: Quantify agent reliability and calculate confidence scores

N8N:

Definition: Workflow automation platform used for orchestrating multi-agent analysis and integration with external services

Alternative: Similar to Zapier but self-hostable

Synthesizer:

Definition: The final GPT-4o instance that integrates all 5 agent analyses, performs HHH evaluation, and generates the executive report

Location: Final node in main workflow

Webhook:

Definition: HTTP endpoint that triggers a workflow when called

Example: POST /webhook/stratai-submit starts analysis workflow

Workflow:

Definition: A sequence of connected nodes in N8N that processes data

Example: Main Strategic Intelligence workflow (14 total workflows)

Acronyms:

API: Application Programming Interface

ARR: Annual Recurring Revenue

CDN: Content Delivery Network

CRUD: Create, Read, Update, Delete

CSV: Comma-Separated Values

DPA: Data Processing Agreement

ELK: Elasticsearch, Logstash, Kibana

GDPR: General Data Protection Regulation

GPU: Graphics Processing Unit

HHH: Helpful, Harmless, Honest

ICP: Ideal Customer Profile

JSON: JavaScript Object Notation

KPI: Key Performance Indicator

LLM: Large Language Model

MVP: Minimum Viable Product

NPS: Net Promoter Score

OAuth: Open Authorization

PDF: Portable Document Format

PII: Personally Identifiable Information

REST: Representational State Transfer

ROI: Return on Investment

RPD: Requests Per Day

RPM: Requests Per Minute

RPO: Recovery Point Objective

RTO: Recovery Time Objective

SLA: Service Level Agreement

SMTP: Simple Mail Transfer Protocol

SQL: Structured Query Language

SSL: Secure Sockets Layer

TAM: Total Addressable Market

TLS: Transport Layer Security

TPM: Tokens Per Minute

TTL: Time To Live

UI: User Interface

UUID: Universally Unique Identifier

WAU: Weekly Active Users

11.2 Troubleshooting Guide

yaml

Common Issues:

Issue: "Webhook returns 404 Not Found"

Symptoms:

- POST request fails
- Error: "Webhook not found"

Diagnosis:

1. Check webhook is activated in N8N
2. Verify URL is correct (including UUID)
3. Check N8N service is running

Solution:

Check N8N service

```
sudo systemctl status n8n
```

Restart if needed

```
sudo systemctl restart n8n
```

Verify webhook in N8N UI

Workflows → Open workflow → Check webhook URL

Test with curl

```
curl -X POST https://your-domain.com/webhook/[uuid] \  
-H "Content-Type: application/json" \  
-d '{"test": "data"}'
```

Issue: "Analysis times out after 60 seconds"

Symptoms:

- Workflow execution stops
- Error: "Execution timed out"
- No response from webhook

Diagnosis:

1. Check N8N timeout settings
2. Verify OpenAI API is responding
3. Check if agents are stuck

Solution:

Increase timeout in .env

```
EXECUTIONS_TIMEOUT=300
```

```
EXECUTIONS_TIMEOUT_MAX=600
```

Restart N8N

```
sudo systemctl restart n8n
```

Check OpenAI status

```
curl https://status.openai.com
```

Review execution logs

N8N UI → Executions → Click failed execution

Issue: "Confidence score is NaN or empty"

Symptoms:

- Report generated but confidence_score is ""
- JavaScript calculation returns NaN

Diagnosis:

1. Check synthesizer output format
2. Verify HHH scores are present
3. Check regex pattern matches

Solution:

Debug in Extract & Validate node

```
console.log('Synthesizer output:', synthesizedReport);
```

```
console.log('Extracted scores:', scores);
```

```
console.log('Base score:', baseScore);
```

```
console.log('Adjustment:', adjustment);
```

Verify HHH format in synthesizer prompt

Should be: [Helpful: X | Harmless: X | Honest: X] = Avg: X.XX%

Test regex pattern

```
const pattern = /Market Data Analysis:\s*\[Helpful:\s*(\d+)\s*\]/;
```

```
const match = synthesizedReport.match(pattern);
```

```
console.log('Regex match:', match);
```

Issue: "Google Doc creation fails"

Symptoms:

- Analysis completes but no Google Doc
- Error: "Invalid credentials"

Diagnosis:

1. Check OAuth token is valid
2. Verify Google Docs API is enabled
3. Check permissions

Solution:

- # Re-authorize Google credentials
- # N8N → Settings → Credentials → Google API
- # Click "Reconnect" → Authorize

- # Verify API is enabled
- # console.cloud.google.com → APIs & Services
- # Check "Google Docs API" is enabled

- # Test manually
- # N8N → Workflows → Main workflow
- # Execute workflow manually
- # Check "Create Google Doc" node output

Issue: "Airtable: Record limit exceeded"

Symptoms:

- Error: "Cannot create record"
- Message: "You've reached your record limit"

Diagnosis:

1. Check current record count
2. Verify Airtable plan limits

Solution:

- # Free tier: 1,200 records per base
- # Upgrade to Plus: \$20/user/month (50,000 records)

- # Clean up old records
- # Airtable → Reports table
- # Filter by date > 90 days ago
- # Select all → Delete

- # Or implement auto-archival workflow

Issue: "Email notifications not sending"

Symptoms:

- Report generated but no email received
- No error in workflow

Diagnosis:

1. Check Gmail API quota
2. Verify email address
3. Check spam folder

Solution:

- # *Verify Gmail credentials*
- # *N8N → Settings → Credentials → Gmail OAuth2*
- # *Click "Reconnect"*

- # *Check Gmail API quotas*
- # *console.cloud.google.com → IAM & Admin → Quotas*
- # *Search "Gmail API"*

- # *Check email node configuration*
- # *Verify "fromEmail" and "toEmail" are correct*

- # *Test email delivery*
- # *Send test email via Gmail web interface*

Issue: "High API costs from OpenAI"

Symptoms:

- Unexpected bill
- Usage dashboard shows high token count

Diagnosis:

1. Check usage patterns
2. Identify inefficient prompts
3. Verify no unauthorized usage

Solution:

- # *Review OpenAI dashboard*
- # *platform.openai.com/usage*

- # *Set usage limits*
- # *platform.openai.com/account/limits*
- # *Set hard limit: \$100/month (or desired amount)*

- # *Optimize prompts*
- # *Reduce system prompt tokens*
- # *Remove unnecessary context*
- # *Use concise language*

- # Check for loops or retries
- # Review N8N execution logs
- # Look for excessive API calls

Issue: "Database connection pool exhausted"

Symptoms:

- Error: "Too many connections"
- Workflows fail to start
- High latency

Diagnosis:

1. Check active connections
2. Verify connection pool settings
3. Look for connection leaks

Solution:

Check current connections

```
sudo -u postgres psql -c \  
"SELECT count(*) FROM pg_stat_activity;"
```

Increase max_connections

```
# /etc/postgresql/14/main/postgresql.conf  
max_connections = 200
```

Restart PostgreSQL

```
sudo systemctl restart postgresql
```

Implement connection pooling

Install PgBouncer (see section 7.2)

Find long-running queries

```
sudo -u postgres psql -c \  
"SELECT pid, query_start, state, query  
FROM pg_stat_activity  
WHERE state != 'idle'  
ORDER BY query_start;"
```

11.3 Support & Resources

yaml

Official Documentation:

N8N:

- Main Docs: <https://docs.n8n.io>
- Community: <https://community.n8n.io>

- **GitHub:** <https://github.com/n8n-io/n8n>

OpenAI:

- **API Docs:** <https://platform.openai.com/docs>
- **Status:** <https://status.openai.com>
- **Community:** <https://community.openai.com>

Anthropic:

- **API Docs:** <https://docs.anthropic.com>
- **Console:** <https://console.anthropic.com>

Google Cloud:

- **Docs API:** <https://developers.google.com/docs/api>
- **Drive API:** <https://developers.google.com/drive/api>
- **Gmail API:** <https://developers.google.com/gmail/api>

Airtable:

- **API Docs:** <https://airtable.com/developers/web/api/introduction>
- **Support:** <https://support.airtable.com>

StratAI Support:

Documentation:

- **GitHub:** <https://github.com/your-org/stratai>
- **Wiki:** <https://github.com/your-org/stratai/wiki>

Contact:

- **Email:** support@stratai.com
- **Slack:** **#stratai-support**
- **Emergency:** +1-XXX-XXX-XXXX (for P1 incidents)

Office Hours:

- **Monday-Friday:** 9 AM - 6 PM EST
- **Response SLA:**
 - * P1 (System down): 1 hour
 - * P2 (Degraded): 4 hours
 - * P3 (Question): 24 hours

Community Resources:

Forums:

- **StratAI Community:** community.stratai.com
- **Discord:** discord.gg/stratai

Social:

- **Twitter:** [@StratAI](https://twitter.com/StratAI)

- **LinkedIn:** linkedin.com/company/stratai

Blog:

- **Tech Blog:** blog.stratai.com
- **Use Cases:** stratai.com/case-studies

11.4 Version History

yaml

Version 1.0.0 (January 2025):

- Initial production release
- 5-agent multi-agent architecture
- HHH evaluation framework
- Confidence scoring system
- 14 complete workflows
- Google Docs integration
- Airtable storage
- Email notifications
- AI chat assistant
- Feedback collection
- Admin analytics

Planned Features (Version 1.1.0 - Q2 2025):

- Advanced filtering in analytics
- Custom agent configurations
- Multi-language support
- Enhanced PDF parsing (tables, images)
- Scheduled analysis (weekly/monthly)
- API rate limiting improvements
- Performance dashboard
- Cost optimization recommendations

Planned Features (Version 2.0.0 - Q3 2025):

- Real-time collaboration
 - Team workspaces
 - Custom report templates
 - Advanced visualizations
 - Mobile app (iOS/Android)
 - SSO integration (SAML, OIDC)
 - Advanced RBAC
 - Audit logging
 - Compliance certifications (SOC 2, ISO 27001)
-

Conclusion

This technical architecture documentation provides a comprehensive guide to the StratAI platform, covering:

1. **System Overview:** High-level architecture and technology stack
2. **Architecture Diagrams:** Visual representations of multi-agent flow and data processing
3. **Component Specifications:** Detailed specs for all 5 agents, synthesizer, and calculators
4. **Data Flow:** Request lifecycle and schema definitions
5. **API Documentation:** All endpoints, webhooks, and integration points
6. **Security & Compliance:** Authentication, encryption, GDPR, and data privacy
7. **Deployment Guide:** Three deployment options with step-by-step instructions
8. **Monitoring & Observability:** Logging, metrics, dashboards, and alerting
9. **Disaster Recovery:** Backup strategies and recovery procedures
10. **Performance Optimization:** Database tuning, API optimization, caching, and load testing

Key Achievements:

- Complete multi-agent strategic intelligence platform
- Production-ready with 14 operational workflows
- Confidence scoring with mathematical rigor
- Scalable architecture supporting 500-50,000+ reports/day
- Comprehensive monitoring and disaster recovery
- GDPR-compliant data handling

Deployment Time Estimates:

- N8N Cloud: 2-4 hours
- Self-hosted: 3-6 hours
- Docker: 2-4 hours

Monthly Operational Costs:

- MVP (100 reports): \$100-200
- Growth (1,000 reports): \$400-600
- Scale (10,000 reports): \$2,000-3,000
- Enterprise (50,000+ reports): \$3,000-5,000